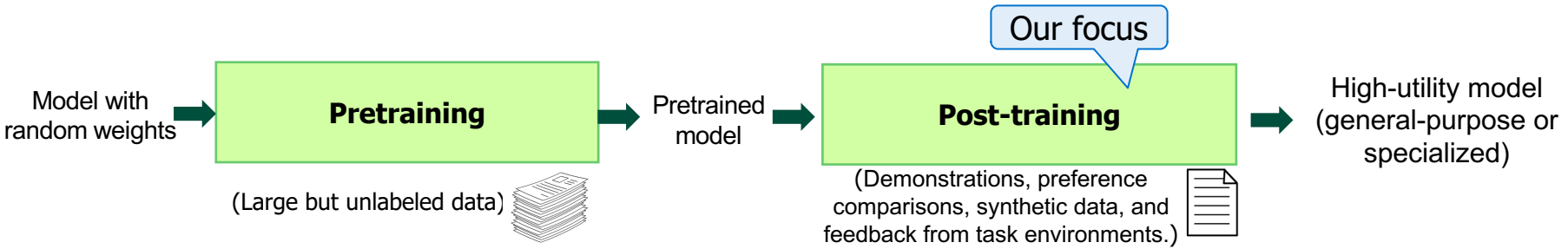
Post-training

CSCI 601-771 (NLP: Self-Supervised Models)

https://self-supervised.cs.jhu.edu/fa2026/

Post-training Language Models

- There is a mismatch between what **pre-trained** models can do and what we want.
- Addressing this gap is the focus of “alignment” research.
- Let’s take a deeper look into what “alignment” is about.



What do people actually do?

Methods SFT RL RLHF PPO GRPO

Feedback Human preferences Verifiable rewards Model-based (reward models, LLM graders, teachers)

Model	Year	Pipeline	Feedback source	Data
Llama 2-Chat	2023	SFT → RLHF (rejection sampling + PPO)	Human preferences → reward models	27.5K SFT
Llama 3 / 3.1	2024	Rejection sampling → SFT → DPO	Human preferences; code execution checks	Heavy synthetic SFT
DeepSeek-R1	2025	SFT → RL (GRPO) → SFT → RL	Verifiable rewards + preference model	800K SFT
Nemotron 3 Ultra	2026	SFT → RL (GRPO) → multi-teacher on-policy distillation + RLHF	Verifiable rewards; preference model; teacher models	10M SFT; 1M RL tasks
Kimi K3	2026	SFT → 9 RL experts → multi-teacher on-policy distillation	Verifiable rewards; LLM graders; teacher models	Synthetic agent tasks
GLM-5.3	2026	Scaled RL only (SAO)	Verifiers on executable environments	Long-horizon environments
MiMo-V2.6-Pro	2026	SFT → mixed RL (GRPO) → on-policy distillation	Test pass × LLM quality checklists	7K+ RL environments

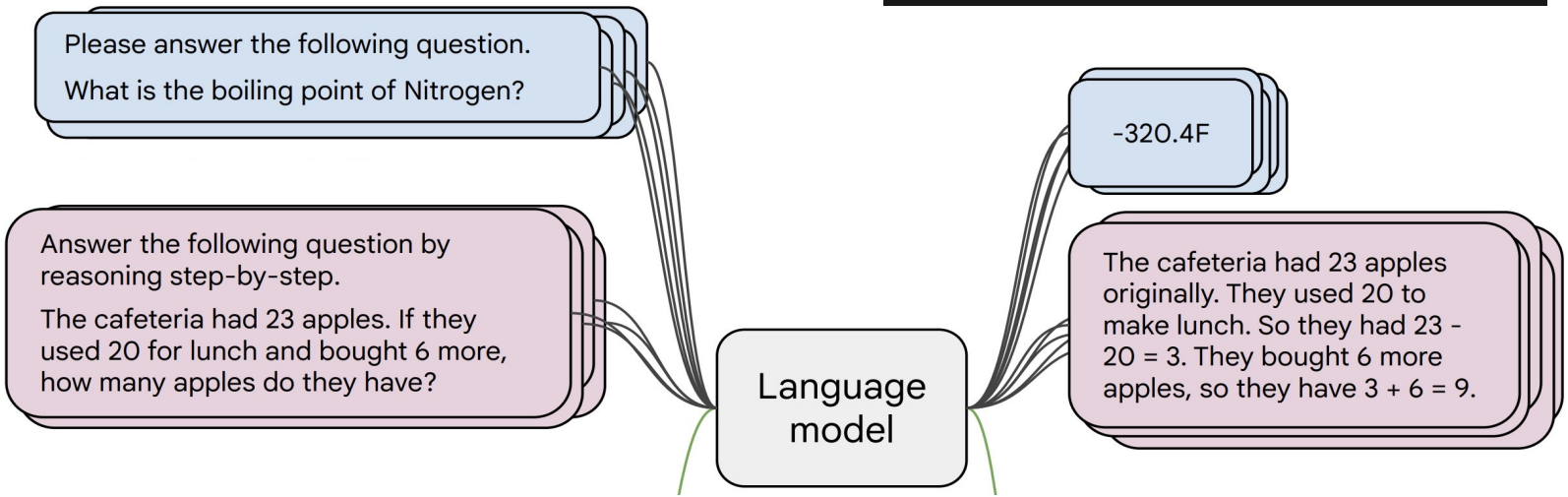
We will see some of these!

Instruction-tuning

Instruction-tuning

- Instruction tuning trains a pretrained LM to generate a demonstrated response conditioned on an instruction and any accompanying context.
- We typically compute the loss on response tokens; instruction and context tokens are provided as conditioning information.

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y)\sim D}\left[\sum_{t=1}^{|y|} \log \pi_\theta(y_t \mid x, y_{<t})\right].$$

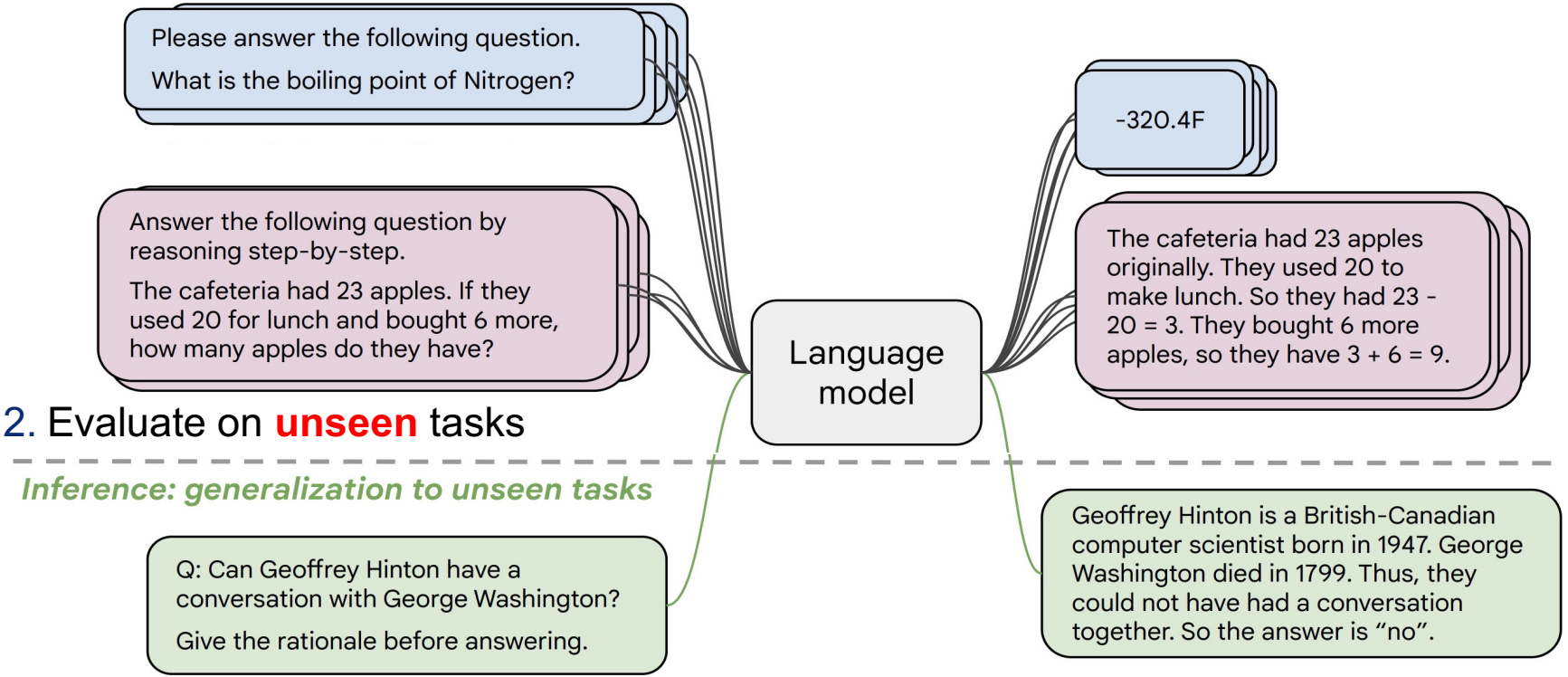


Instruction-tuning

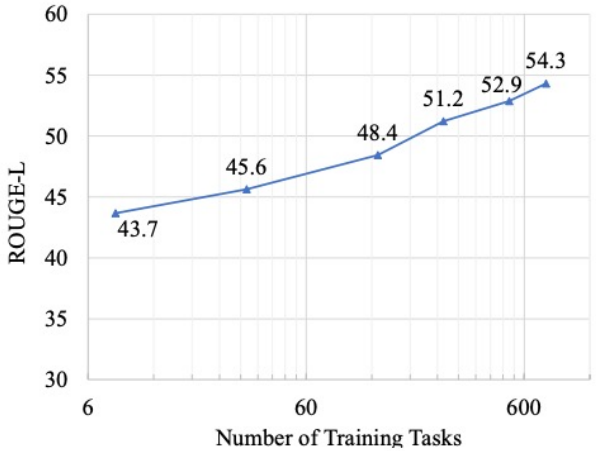
[Weller et al. 2020. Mishra et al. 2021; Wang et al. 2022, Sanh et al. 2022; Wei et al., 2022, Chung et al. 2022, many others]

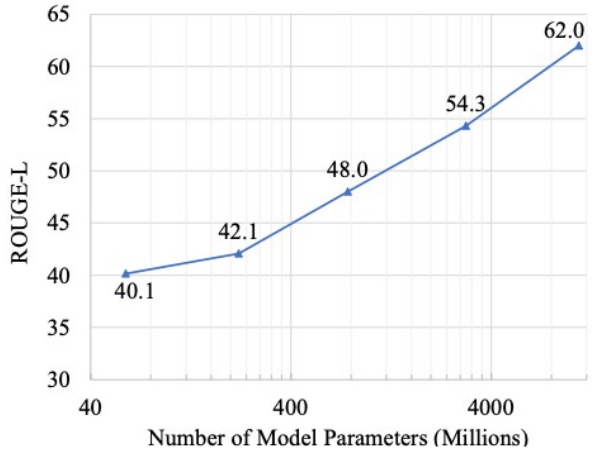
- Task generalization means evaluating on task types excluded from training.

1. Collect examples of (instruction, output) pairs across many tasks and finetune LM



Scaling Instruction-Tuning





Linear growth of model performance
with exponential increase in observed tasks and model size.

Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks, Wang et al. 2022

Cross-Task Generalization via Natural Language Crowdsourcing Instructions (Mishra et al., 2022)

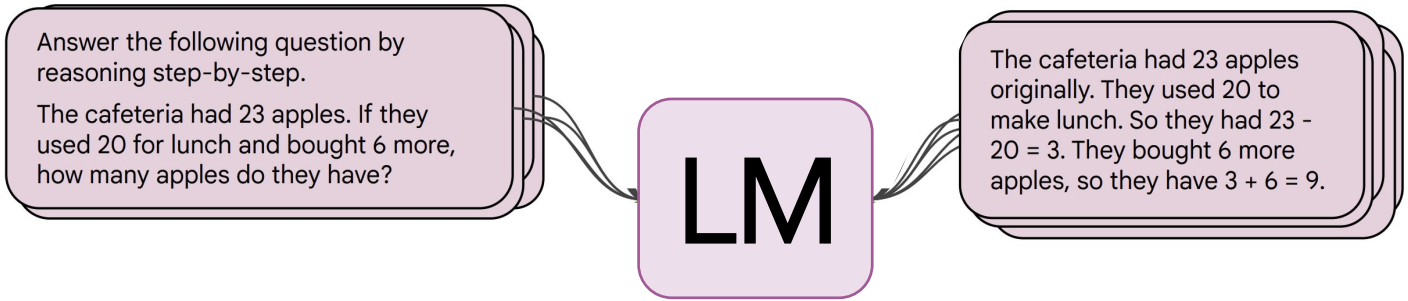
Pulse check

- **Q1:** Does instruction tuning require a different objective from pretraining?
- **A1:** Mostly no—both use next-token prediction. SFT typically computes loss on response tokens, conditioned on the instruction and context.

- **Q2:** If a model succeeds on new examples of familiar tasks, has it demonstrated unseen-task generalization?
- **A2:** No. Unseen-task evaluation holds out entire task types, not just individual examples.

Limits of Instruction-Tuning

1. Difficult to collect diverse data.
2. Resulting models may not be good at open-ended generation tasks.
   - SFT increases the likelihood of demonstrated responses. => Its effectiveness therefore depends strongly on the quality and coverage of the demonstrations.



Limits of Instruction-Tuning

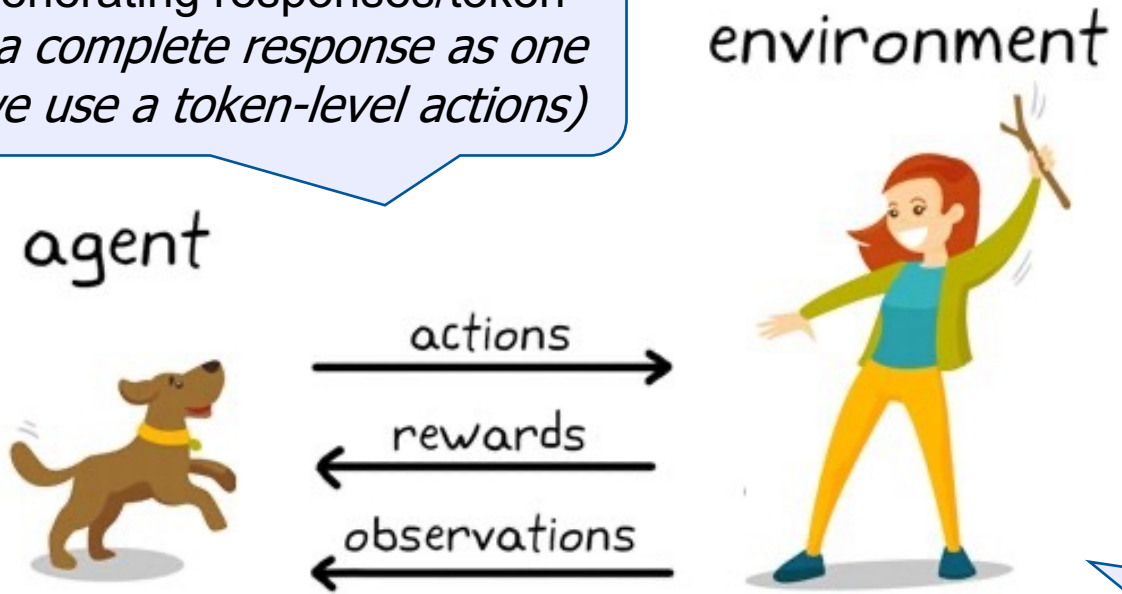
1. Difficult to collect diverse data.
2. Resulting models may not be good at open-ended generation tasks.
   - SFT increases the likelihood of demonstrated responses. => Its effectiveness therefore depends strongly on the quality and coverage of the demonstrations.
3. SFT can encourage unsupported factual answers when demonstrations require knowledge the model does not reliably possess.
   - Labeled data is collected agnostic to the LM's knowledge (off-policy) => there might be a mismatch between labeled data and LM knowledge.

Does Fine-Tuning LLMs on New Knowledge Encourage Hallucinations?

Reinforcement Learning

Reinforcement Learning: Intuition

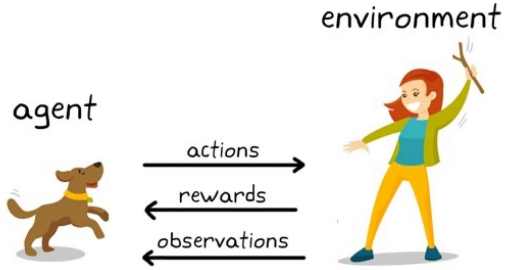
Action here: generating responses/token
(for now, treat a complete response as one action. Later, we use a token-level actions)



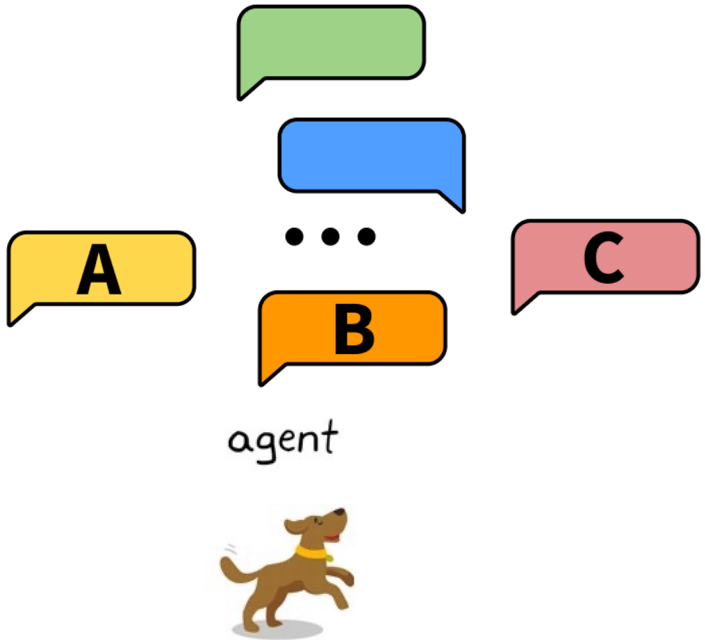
[figure credit]

Reward here: whether humans liked the generation (sequence of actions=tokens)

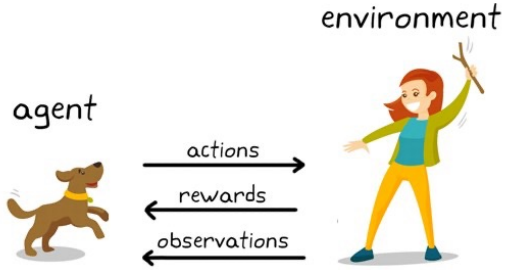
Intuition



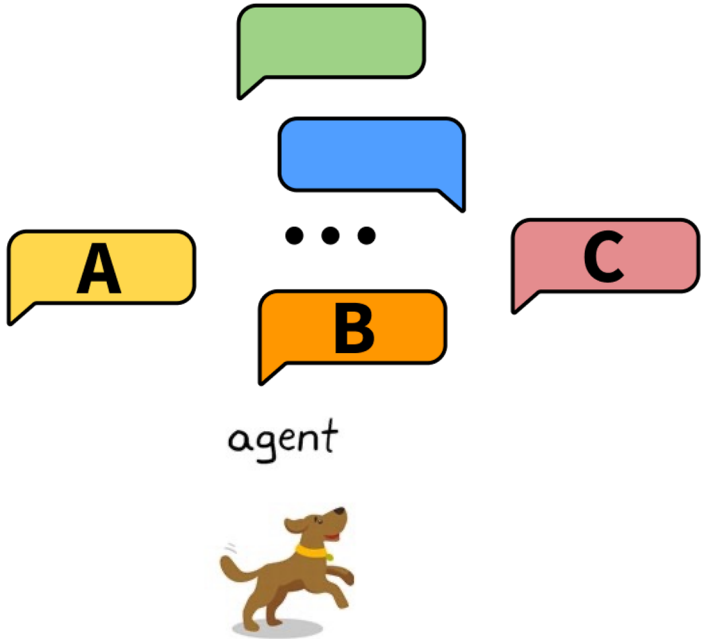
- The LM generates candidate responses to a conversation.
- Human compare those candidates

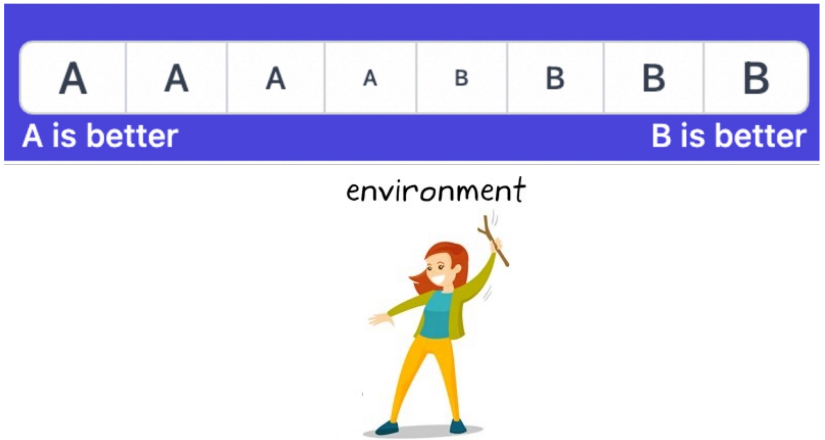


Intuition



- Scoring interface: Likert scale or rankings
  - E.g., each annotation identifies a preferred vs. a rejected response to the same prompt.
  - Preferences depend on the annotation rubric and may differ across annotators.





Reinforcement Learning: Formalism

- **State s_t :** the prompt and generated prefix.
- **Action a_t :** the next token.
- **Episode:** the agent (policy) generates actions (tokens) until a stopping condition.
- **Transition:** append the generations to the prefix to obtain a new state of the world.
- **Reward:** The environment rewards the action, only after the response is complete.
- We need to figure out: (1) reward function and (2) the policy function



s_t: state (context)
r_t: reward
a_t: actions (new sentences)
p_θ: policy (decision-maker)

[Fig credit: Nate Lambert]

Reinforcement Learning from Human Feedback



- Imagine a reward function: $R(s; \text{prompt}) \in \mathbb{R}$ for any output s to a prompt.
- The reward is higher when humans prefer the output.
- Good generation is equivalent to finding reward-maximizing outputs:

Expected reward over samples from our policy (generative model)

$$\mathbb{E}_{\hat{s} \sim p_\theta}[R(\hat{s}; \text{prompt})]$$

$p_\theta(s)$ is a pre-trained model with params θ we would like to optimize (policy function)

- On the notation:
  - "$\mathbb{E}$" here is an expectation under probability distribution (i.e., average in practice).
  - "~" indicates sampling from a given distribution.

Reinforcement Learning from Human Feedback



- Imagine a reward function: $R(s; \text{prompt}) \in \mathbb{R}$ for any output s to a prompt.
- The reward is higher when humans prefer the output.
- Good generation is equivalent to finding reward-maximizing outputs:

$$\mathrm{J}(\theta) = \mathbb{E}_{\text{prompt}\sim D}\mathbb{E}_{\hat{s}\sim p_\theta}[R(\hat{s}; \text{prompt})]$$

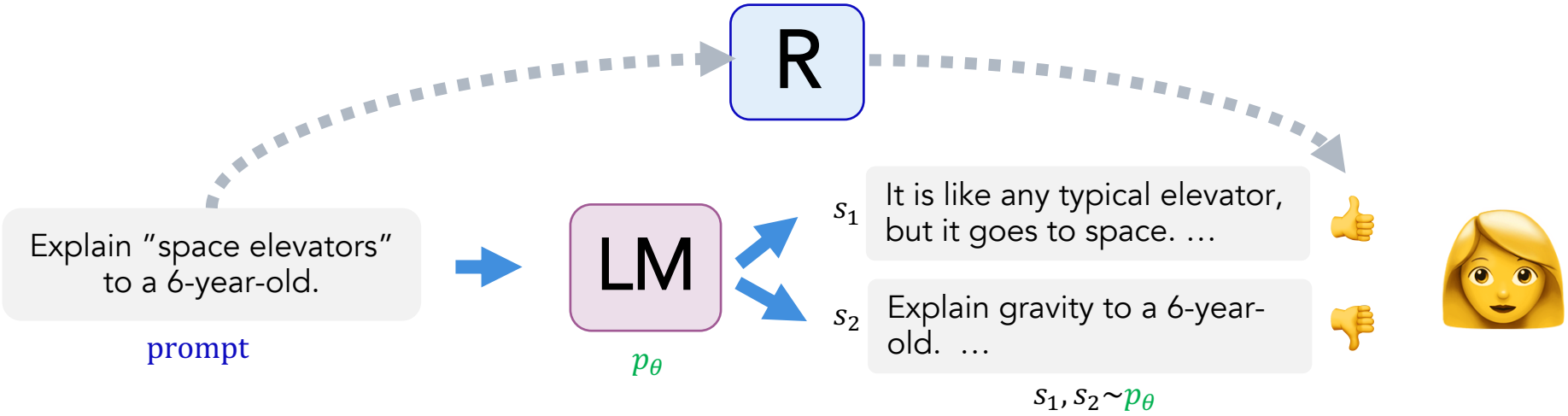
- We optimize **average** reward over a *distribution of prompts* and *responses sampled from the current policy*.

- What remains here:
  - (1) Estimate the reward function $R(s; \text{prompt})$.
  - (2) Find the best generative model p_θ that maximizes the expected reward:

Step 1: Estimating the Reward R



- Humans look at alternative generations and indicate their preference.



Which of these two responses is more helpful?

What are the steps for making a simple cake?

1. *Preheat oven to 350°F (175°C).*
2. *Grease and flour a cake pan.*
3. *In a bowl, combine 2 cups flour, 1.5 cups sugar, 3.5 tsp baking powder, and a pinch of salt.*
4. *Add 1/2 cup butter, 1 cup milk, and 2 tsp vanilla; mix well.*
5. *Beat in 3 eggs, one at a time.*
6. *Pour batter into the pan.*
7. *Bake for 30-35 minutes or until a toothpick comes out clean.*
8. *Let cool, then frost or serve as desired.*

What are the steps for making a simple cake?

1. *Warm up the oven.*
2. *Grease a cake pan.*
3. *Blend dry ingredients in a bowl.*
4. *Incorporate butter, milk, and vanilla.*
5. *Mix in the eggs.*
6. *Pour into the prepared pan.*
7. *Bake until golden brown.*
8. *Add frosting if desired.*

[Example from Eric Mitchell]

Step 1: Estimating the Reward R

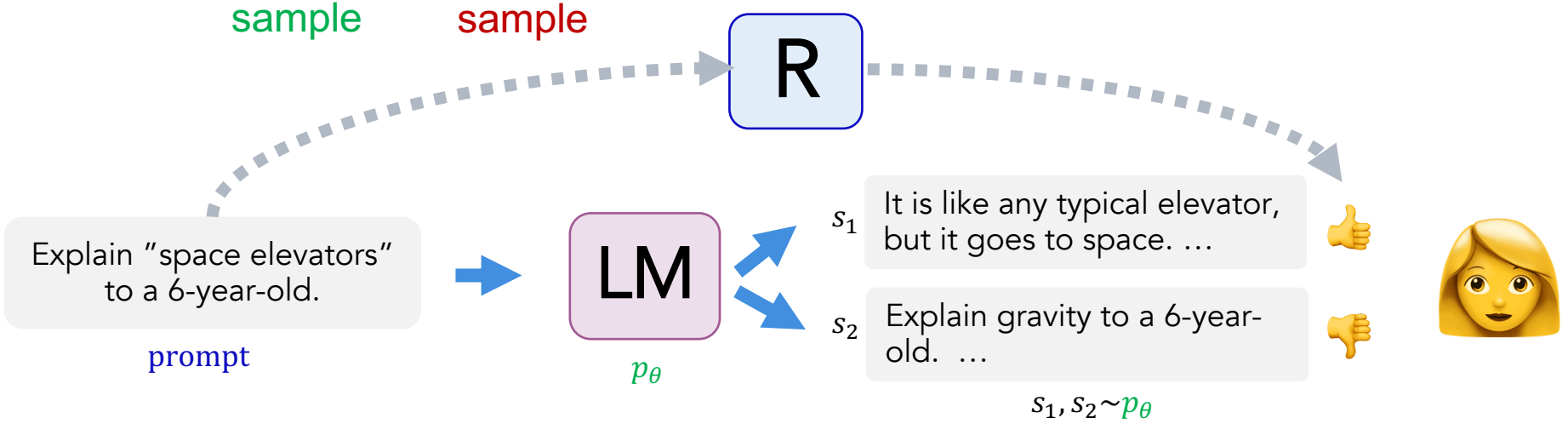


- A larger score difference assigns a higher probability to the observed preference.

$$P(s^+ > s^- | \text{prompt}) = \sigma\big(R(s^+; \text{prompt}) - R(s^-; \text{prompt})\big)$$

"winning" sample

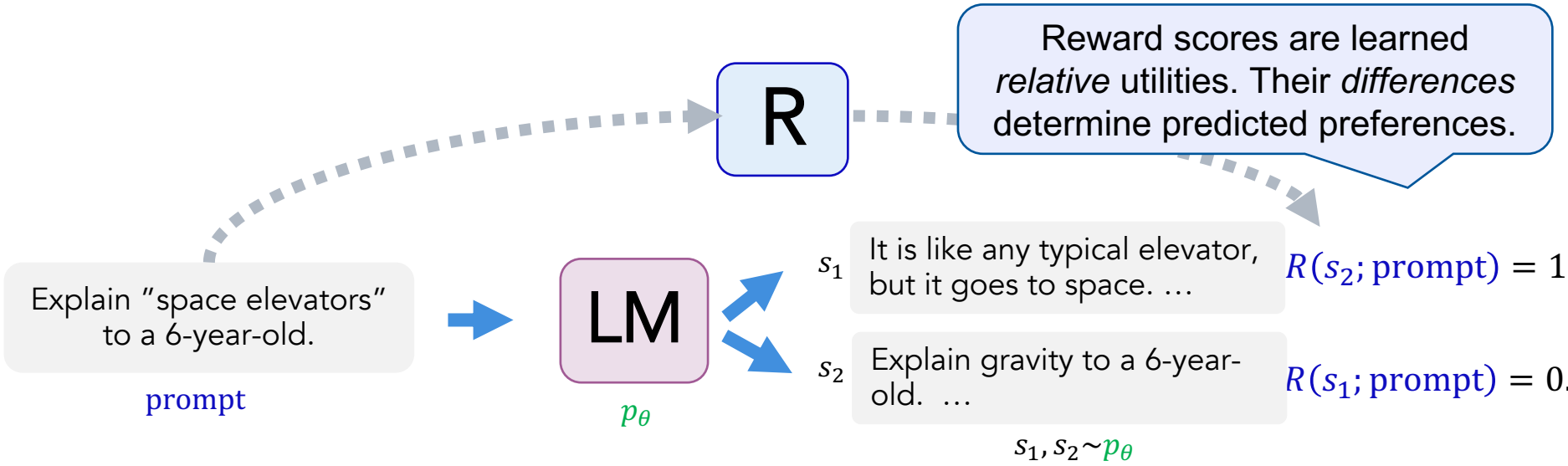
"losing" sample



Step 1: Estimating the Reward R

- Given the human ratings, we can train the reward model:

$$J_{RM}(\phi) = -\mathbb{E}_{\text{prompt}\sim D}\mathbb{E}_{(s^+,s^-)\sim}\left[\log\sigma\left(R(s^+;\text{prompt}) - R(s^-;\text{prompt})\right)\right]$$



Step 1:
Estimating the Reward R

```
loss_fn = nn.BCEWithLogitsLoss()  # For preference classification

for epoch in range(epochs):
    model.train()
    total_loss = 0
    for a, b, label in dataloader:
        a, b, label = a.to(device), b.to(device), label.to(device).float()

        reward_a = model(a)
        reward_b = model(b)

        logits = reward_a - reward_b
        loss = loss_fn(logits, label)
```

Where did the “sigmoid” go??

https://docs.pytorch.org/docs/2.15/generated/torch.nn.BCEWithLogitsLoss.html

Pulse check

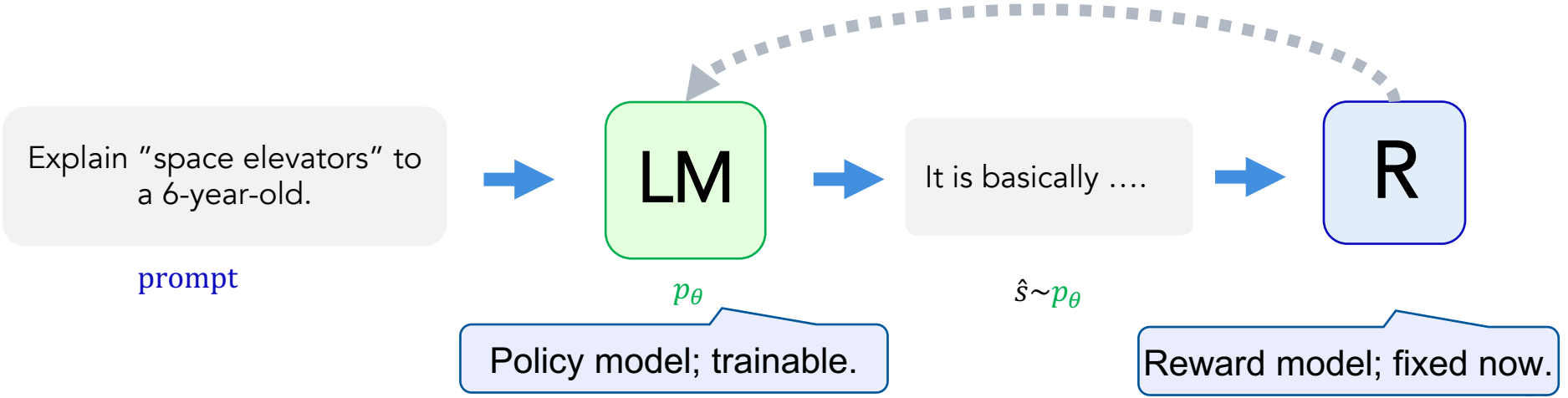
- **Q:** Do we need humans to assign numerical rewards to train a reward model?
- **A:** No. Pairwise preferences are enough: train the model to score the preferred response higher than the rejected response.

Step 2: Optimizing the Policy Function



- **Policy function** := The model that makes decisions (here, generates responses)
- How do we change our LM parameters θ to maximize this objective?

$$J(\theta) = \mathbb{E}_{\text{prompt}\sim D}\,\mathbb{E}_{\hat{s}\sim p_\theta}[R(\hat{s};\text{prompt})]$$



Step 2: Optimizing the Policy Function



- **Policy function** := The model that makes decisions (here, generates responses)
- How do we change our LM parameters θ to maximize this objective?

$$\mathrm{J}(\theta) = \mathbb{E}_{\text{prompt}\sim D}\,\mathbb{E}_{\hat{s}\sim p_\theta}[R(\hat{s};\text{prompt})]$$

- Let's try doing gradient ascent!

$$\theta_{t+1} \leftarrow \theta_t + \alpha\,\nabla_{\theta_t}\mathrm{J}(\theta_t)$$

How do we estimate the gradient of this expectation?

Notice that R is not directly dependent on θ. (You can't compute its grad with respect to θ)

- The response is sampled through discrete token choices, so ordinary backprop cannot differentiate through the sampled text.
- So how do we estimate these gradients??

Policy Gradient [Williams, 1992]



- How do we change our LM parameters θ to maximize this objective?

$$\mathrm{J}(\theta) = \mathbb{E}_{\text{prompt}\sim D}\,\mathbb{E}_{\hat{s}\sim p_\theta}\left[R(\hat{s};\text{prompt})\right]$$

- Let's try doing gradient ascent!

$$\theta_{t+1} \leftarrow \theta_t + \alpha\,\nabla_{\theta_t}\mathrm{J}(\theta_t)$$

- With a bit of math, this can be approximated as Monte Carlo samples from $p_\theta(s)$:

"hat" because we're estimating gradient using finite samples

$$\widehat{\nabla_{\theta_t}\mathrm{J}(\theta_t)} \propto \sum_{\text{prompt}\sim D}\ \sum_{s_i\sim p_\theta(\text{prompt})} R(s_i;\text{prompt})\,\nabla_\theta \log p_\theta(s_i;\text{prompt})$$

- This is "policy gradient", an approach for estimating and optimizing this objective.

Bonus

Derivations

(check it later in your own time!)



- Let's compute the gradient:

Def. of "expectation"

Gradient distributes over sum

$$\nabla_\theta \mathbb{E}_{s\sim p_\theta(s)}[R(s;p)] = \nabla_\theta \sum_s p_\theta(s) R(s;p) = \sum_s R(s;p) \,.\, \nabla_\theta p_\theta(s)$$

- Log-derivative trick $\nabla_\theta p_\theta(s) = p_\theta(s) \,.\, \nabla_\theta \log p_\theta(s)$ to turn sum back to expectation:

Log-derivative trick

$$\nabla_\theta \mathbb{E}_{s\sim p_\theta(s)}[R(s;p)] = \sum_s R(s;p)\, p_\theta(s)\, \nabla_\theta \log p_\theta(s) = \mathbb{E}_{s\sim p_\theta(s)}[R(s;p)\, \nabla_\theta \log p_\theta(s)]$$

- Approximate this expectation with Monte Carlo samples from $p_\theta(s)$:

$$\nabla_\theta \mathbb{E}_{s\sim p_\theta(s)}[R(s;p)] \approx \frac{1}{n}\sum_{i=1}^{n} R(s;p)\, \nabla_\theta \log p_\theta(s)$$

Policy Gradient [Williams, 1992]



- This gives us the following update rule:

Note, $R(s; \text{prompt})$ could be any arbitrary, non-differentiable reward function that we design.

$$\theta_{t+1} \leftarrow \theta_t + \alpha \frac{1}{|\text{samples}| \times |\text{prompts}|} \sum_{\text{p} \in \text{prompts}} \sum_{s_i \sim p_\theta(\text{p})} R(s_i; \text{p}) \nabla_\theta \log p_\theta(s_i; \text{p})$$

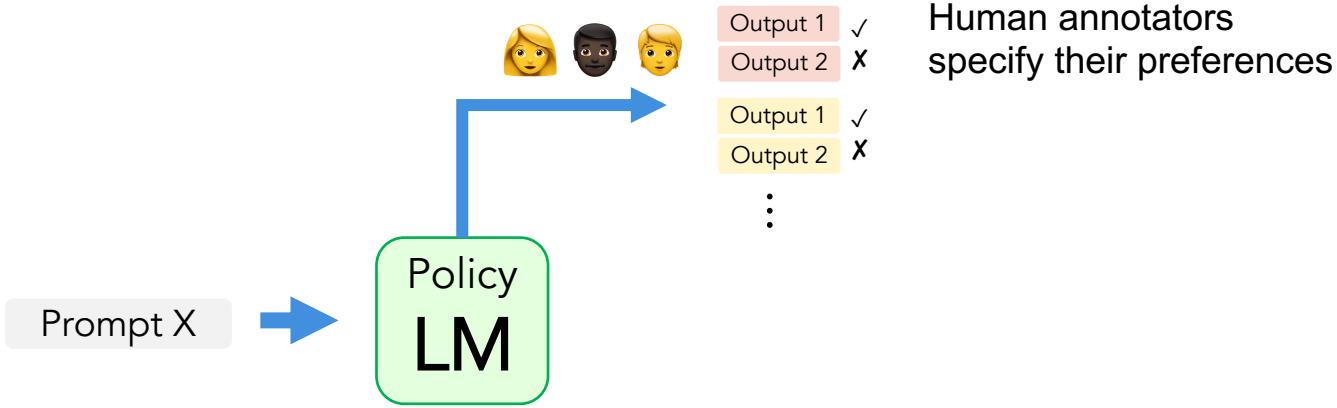
- If $R(s; p)$ is large, we take proportionately large steps to maximize $p_\theta(s)$
- If $R(s; p)$ is small, we take proportionately small steps to maximize $p_\theta(s)$

This is why it's called "reinforcement learning":
we reinforce good actions, increasing the chance they happen again.

Putting it Together



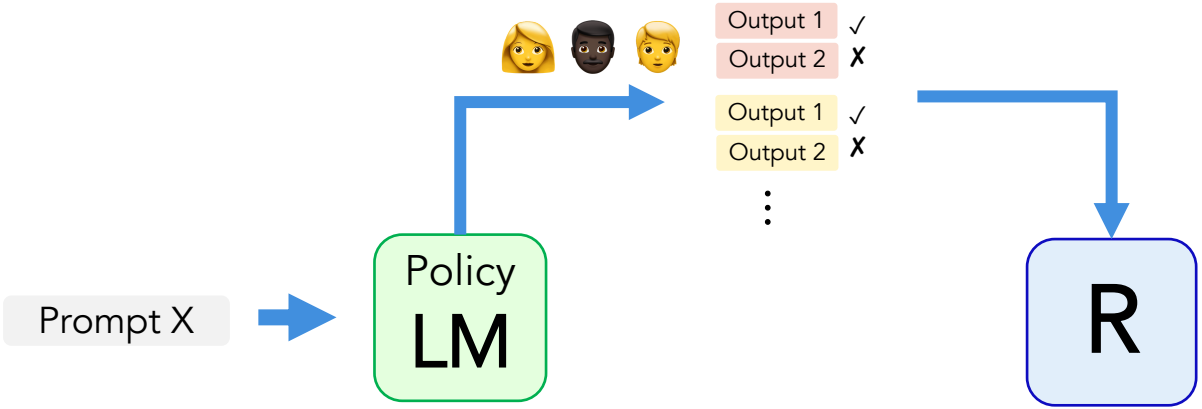
- First collect a dataset of human preferences
  - Present multiple outputs to human annotators and ask them to rank the output based on preferability



Putting it Together (2)



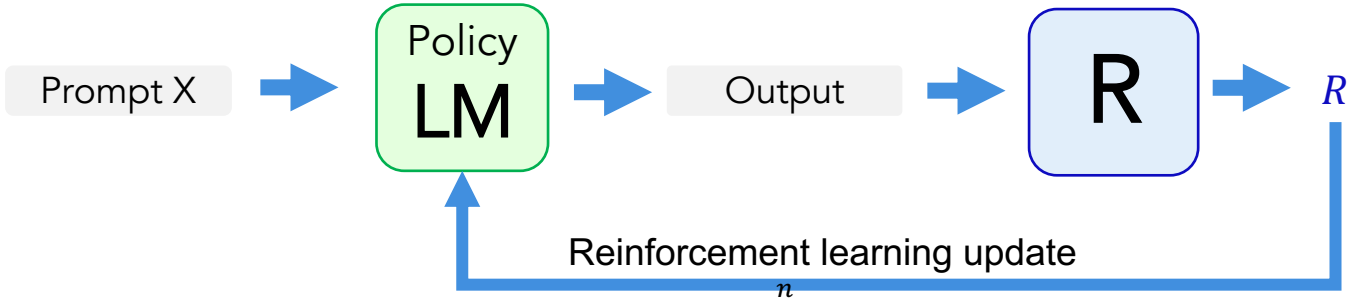
- Using this data, we can train a reward model
  - The reward model returns a scalar reward which should numerically represent the human preference.



Putting it Together (3)



- We want to learn a policy (a Language Model) that optimizes against the reward model

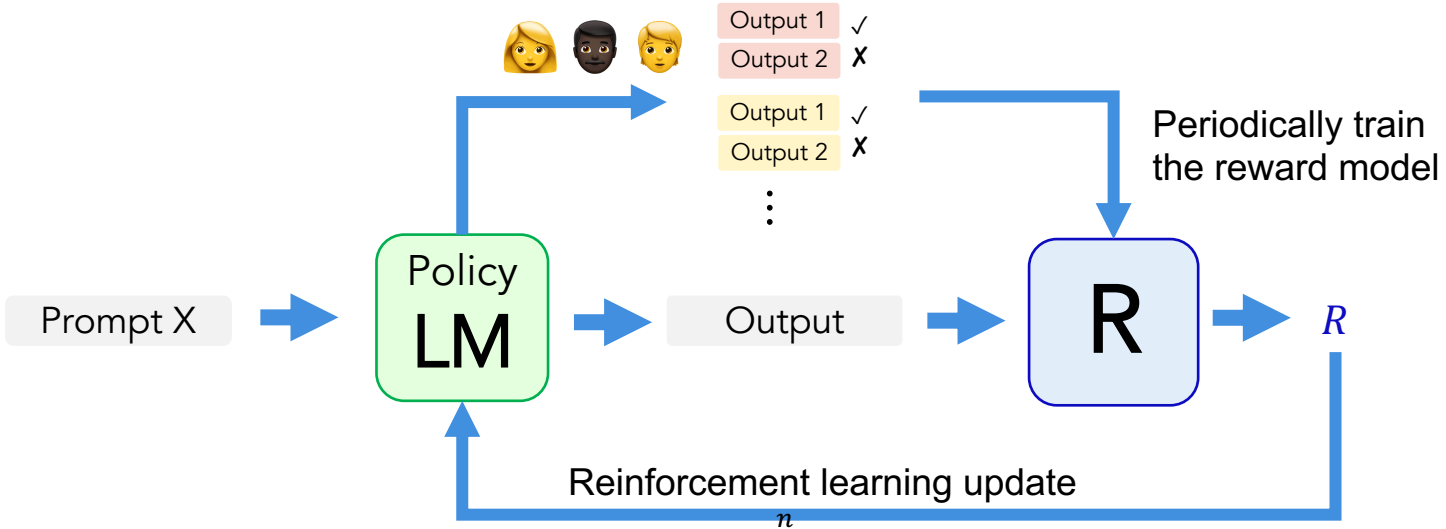


$$\theta_{t+1} \leftarrow \theta_t + \alpha \sum_{i=1}^{n} R(s_i; \mathrm{p})\, \nabla_\theta \log p_\theta(s_i; \mathrm{p})$$

Putting it Together (4)



- Periodically train the reward model with more samples and human feedback

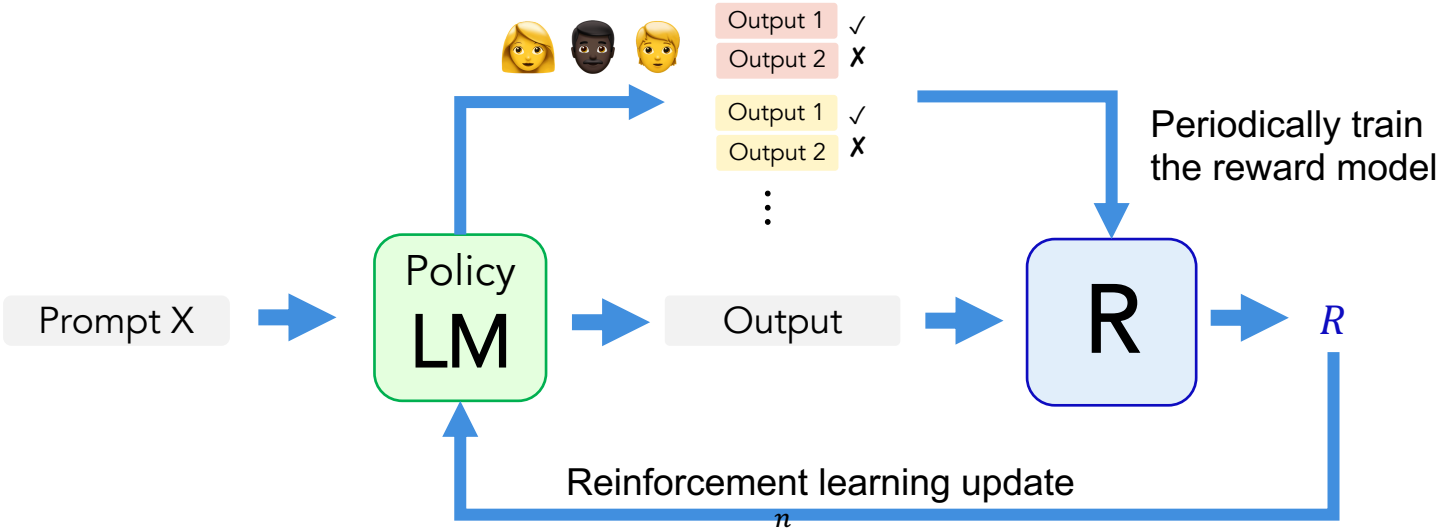


$$\theta_{t+1} \leftarrow \theta_t + \alpha \sum_{i=1}^{n} R(s_i; \mathrm{p}) \nabla_\theta \log p_\theta(s_i; \mathrm{p})$$

One missing ingredient



- It turns out that this approach doesn't quite work. (Any guesses why?)
  - The policy will learn to "cheat".

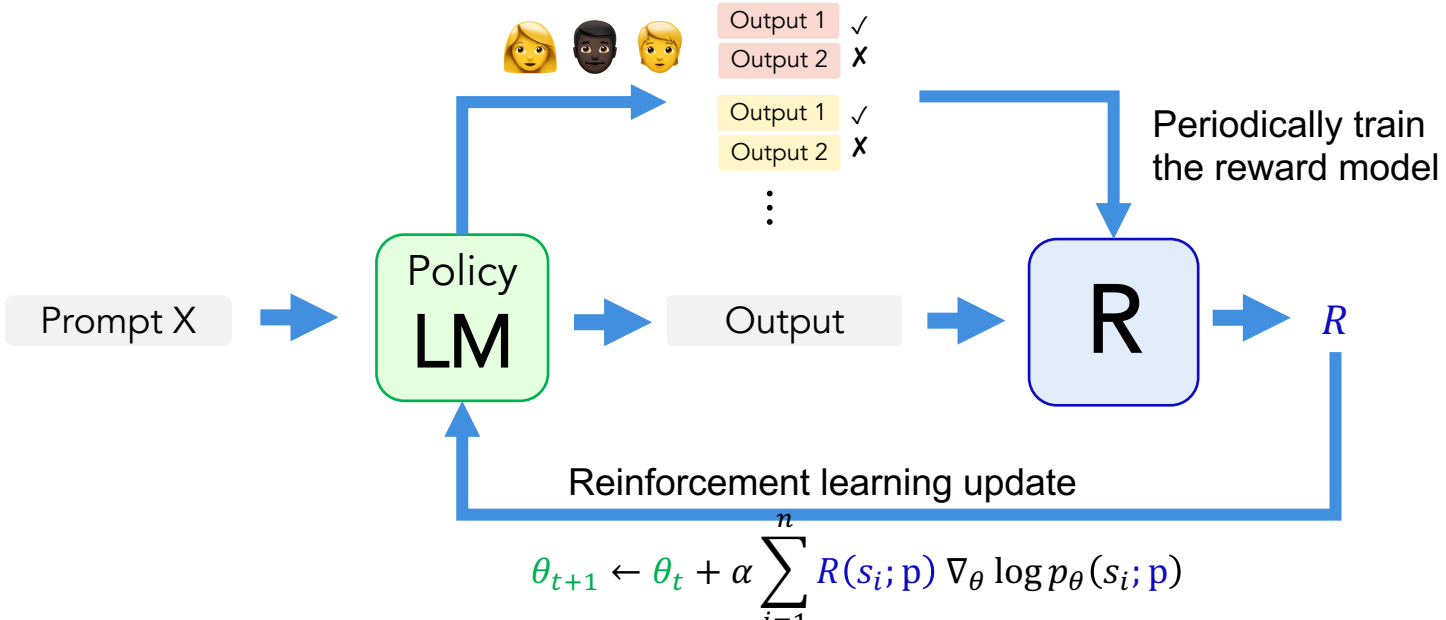


$$\theta_{t+1} \leftarrow \theta_t + \alpha \sum_{i=1}^{n} R(s_i; \mathrm{p}) \nabla_\theta \log p_\theta(s_i; \mathrm{p})$$

One missing ingre

How do you resolve this? 🤔

- Will learn to produce an output that would get a **high** reward but is **gibberish** or **irrelevant** to the prompt.
- Note, since $R(s; p)$ is trained on natural inputs, it may not generalize to unnatural inputs.



Regularizing with Pre-trained Model

- **Solution:** add a penalty term that penalizes too much deviations from the distribution of the pre-trained LM.

$$\hat{R}(s;p) \coloneqq R(s;p) - \beta \log \left(\frac{p_\theta^{RL}(s)}{p^{PT}(s)} \right)$$

- This prevents the policy model from diverging too far from the pretrained model.
  - $p^{RL}(s) >> p^{PT}(s)$: Pay an *explicit* price
  - $p^{RL}(s) << p^{PT}(s)$: Sampling s becomes unlikely
- The above regularization is *equivalent* to adding a KL-divergence regularization term.

Putting it All Together:

RLHF as a Basic Policy Gradient

1. Select a pre-trained generative model p_θ^{RL} as your base: $p_\theta^{PT}(s)$
2. Build a reward model $R(s;p)$ that produces scalar rewards for outputs, trained on a dataset of human comparisons
3. Regularize the reward function:

$$\hat{R}(s;p) \coloneqq R(s;p) - \beta \log\left(\frac{p^{RL}(s)}{p^{PT}(s)}\right)$$

4. Iterate:
   1. Fine-tune the policy $p_\theta^{RL}(s)$ to maximize our reward model $R(s;p)$

$$\theta_{t+1} \leftarrow \theta_t + \alpha \frac{1}{n} \sum_{i=1}^{n} \hat{R}(s;p)\, \nabla_\theta \log p_\theta^{RL}(s)$$

   2. Occasionally repeat repeat 2-3 to update the reward model.

Stiennon et al. Learning to summarize from human feedback, 2020

The overall recipe 👨‍🍳



Pulse Check

- **Q1:** Do humans score every response during policy optimization?
- **A1:** Usually no. Human comparisons train a reward model, which then scores newly generated responses.

- **Q2:** How can we optimize a policy using a non-differentiable reward?
- **A2:** Policy gradients use rewards to weight gradients of the sampled responses' log-probabilities.

- **Q3:** Why penalize divergence from a reference policy?
- **A3:** To discourage excessive distribution shift and reduce exploitation of the reward model's weaknesses.

- Lots of cool MOE + RL stuff: https://cursor.com/resources/Composer2.pdf

What is the Standard?

Methods SFT RL RLHF PPO GRPO

Feedback Human preferences Verifiable rewards Model-based (reward models, LLM graders, teachers)

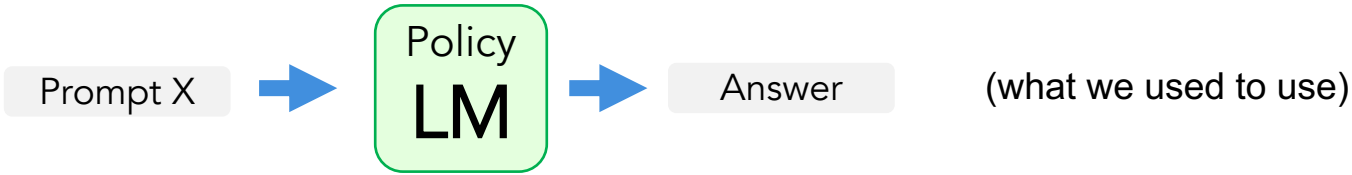
We just saw

Model	Year	Pipeline	source	Data
Llama 2-Chat	2023	SFT → RLHF (rejection sampling + PPO)	Human preferences → reward models	27.5K SFT
Llama 3 / 3.1	2024	Rejection sampling → SFT → DPO	Human preferences; code execution checks	Heavy synthetic SFT
DeepSeek-R1	2025	SFT → RL (GRPO) → SFT → RL	Verifiable rewards + preference model	800K SFT
Nemotron 3 Ultra	2026	SFT → RL (GRPO) → multi-teacher on-policy distillation + RLHF	Verifiable rewards; preference model; teacher models	10M SFT ; 1M RL tasks
Kimi K3	2026	SFT → 9 RL experts → multi-teacher on-policy distillation	Verifiable rewards; LLM graders; teacher models	Synthetic agent tasks
GLM-5.3	2026	Scaled RL only (SAO)	Verifiers on executable environments	Long-horizon environments
MiMo-V2.6-Pro	2026	SFT → mixed RL (GRPO) → on-policy distillation	Test pass × LLM quality checklists	7K+ RL environments

GRPO: Forthcoming

Building our next alignment algorithm

- **Modification 1: thinking**
  - We will “think” (chains of thoughts) before generating answers



Building our next alignment algorithm

- **Modification 1: thinking**
  - We will “think” (chains of thoughts) before generating answers



Building our next alignment algorithm

- **Modification 1: thinking**
  - We will “think” (chains of thoughts) before generating answers



Building our next alignment algorithm

- **Modification 2: drop the parametric rewards, whenever possible.**
  - We used to score the whole output. But it may be enough to score the final answer.
  - But scoring the final answer may not need another model. E.g.,
    - Math problems: check the final answer
    - Programming problems: check the unit tests.

Building our next alignment algorithm

- **Modification 2: drop the parametric rewards, whenever possible.**

2.2.2. Reward Modeling

The reward is the source of the training signal, which decides the optimization direction of RL. To train DeepSeek-R1-Zero, we adopt a rule-based reward system that mainly consists of two types of rewards:

- **Accuracy rewards**: The accuracy reward model evaluates whether the response is correct. For example, in the case of math problems with deterministic results, the model is required to provide the final answer in a specified format (e.g., within a box), enabling reliable rule-based verification of correctness. Similarly, for LeetCode problems, a compiler can be used to generate feedback based on predefined test cases.
- **Format rewards**: In addition to the accuracy reward model, we employ a format reward model that enforces the model to put its thinking process between '<think>' and '</think>' tags.

We do not apply the outcome or process neural reward model in developing DeepSeek-R1-Zero, because we find that the neural reward model may suffer from reward hacking in the large-scale reinforcement learning process, and retraining the reward model needs additional training resources and it complicates the whole training pipeline.

```python
# Reward functions
def correctness_reward_func(prompts, completions, answer, **kwargs) -> list[fl
    responses = [completion[0]['content'] for completion in completions]
    q = prompts[0][-1]['content']
    extracted_responses = [extract_xml_answer(r) for r in responses]
    print('-'*20, f"Question:\n{q}", f"\nAnswer:\n{answer[0]}", f"\nResponse:\
    return [2.0 if r == a else 0.0 for r, a in zip(extracted_responses, answer

def int_reward_func(completions, **kwargs) -> list[float]:
    responses = [completion[0]['content'] for completion in completions]
    extracted_responses = [extract_xml_answer(r) for r in responses]
    return [0.5 if r.isdigit() else 0.0 for r in extracted_responses]

def strict_format_reward_func(completions, **kwargs) -> list[float]:
    """Reward function that checks if the completion has a specific format."""
    pattern = r"^<reasoning>\n.*?\n</reasoning>\n<answer>\n.*?\n</answer>\n$"
    responses = [completion[0]["content"] for completion in completions]
    matches = [re.match(pattern, r, flags=re.DOTALL) for r in responses]
    return [0.5 if match else 0.0 for match in matches]
```

Building our next alignment algorithm

- **Modification 3: off-policy samples**
  - What we used to do: $\hat{\theta} = \text{argmax}_\theta \, \mathbb{E}_{a_t \sim \pi_\theta} [R_t]$

$$\theta_{t+1} \leftarrow \theta_t + \alpha g^{\text{PG}} \qquad g^{\text{PG}} = \mathbb{E}_{a_t \sim \pi_\theta} [\nabla_\theta \log \pi_\theta (a_t | s_t) \, R_t]$$

  - Generating rollouts is expensive (full forward pass for *each token*).
  - Naively, *on-policy* training throws away each rollout *after one gradient step*.
  - Instead: re-use rollouts from a slightly "old[er]" policy π_{old} .
  - But wait … $\mathbb{E}_{a_t \sim \pi_\theta} [f] \neq \mathbb{E}_{a_t \sim \pi_{\text{old}}} [f]$
  - We correct this mismatch by using *importance sampling*:

$$\mathbb{E}_{a_t \sim \pi_\theta} [f] = \mathbb{E}_{a_t \sim \pi_{\text{ref}}} \left[\frac{\pi_\theta (a_t | s_t)}{\pi_{\text{old}} (a_t | s_t)} f \right]$$

Building our next alignment algorithm

- **Modification 3: off-policy samples**
  - What we used to do: $\hat{\theta} = \text{argmax}_\theta \, \mathbb{E}_{a_t \sim \pi_\theta} [R_t]$

$$\theta_{t+1} \leftarrow \theta_t + \alpha g^{\text{PG}} \qquad g^{\text{PG}} = \mathbb{E}_{a_t \sim \pi_\theta} [\nabla_\theta \log \pi_\theta (a_t|s_t) \, R_t]$$

  - Now our modified update rule with slightly off-policy samples:

$$\hat{\theta} = \text{argmax}_\theta \, \mathbb{E}_{a_t \sim \pi_\theta} [R_t] = \text{argmax}_\theta \, \mathbb{E}_{a_t \sim \pi_{\text{old}}} \left[\frac{\pi_\theta (a_t|s_t)}{\pi_{\text{old}} (a_t|s_t)} R_t \right]$$

$$g^{\text{PG}} = \mathbb{E}_{a_t \sim \pi_{\text{old}}} \left[\nabla_\theta \log \pi_\theta (a_t|s_t) . \frac{\pi_\theta (a_t|s_t)}{\pi_{\text{old}} (a_t|s_t)} . R_t \right]$$

Building our next alignment algorithm

- **Modification 4: Use ”advantages” instead of reward values**
  - ”Advantage” is essentially a normalized reward.
  - The goal of this normalization is to *reduce the variance* of our estimates.
  - The exact computation (which depends on the algorithm) will be shown later.
  - For now, wherever we see R_t we will replace it with A_t.

$$\hat{\theta} = \text{argmax}_{\theta}\, \mathbb{E}_{a_t \sim \pi_{\text{old}}} \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)} A_t \right]$$

Building our next alignment algorithm

- **Modification 5: "Clipped" objective**
  - KL regularization was important to prevent over-optimization:

$$L(\theta) = \mathbb{E}_{a_t \sim \pi_{\text{old}}} \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)} A_t + \text{KL}[\pi_{\text{old}}(.|a_t), \pi_\theta(.|a_t)] \right]$$

  - This objective can be [somewhat] equivalently stated in clipped form:

$$L^{\text{clip}}(\theta) = \mathbb{E}_{a_t \sim \pi_{\text{old}}} [\min(r_t(\theta)A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)]$$

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

This looks complex. Let's make sense of it.

Schulman et al. 2017, Proximal Policy Optimization Algorithms

Building our next alignment algorithm

- **Modification 5: "Clipped" objective**
  - This objective can be [somewhat] equivalently stated in clipped form:

$$L^{\text{clip}}(\theta) = \mathbb{E}_{a_t \sim \pi_{\text{old}}}\left[\min(r_t(\theta)A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)\right] \qquad r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

- It's the prob of acting a_t at state s_t in the current policy divided by the previous one
  - If $r_t(\theta) > 1$, then the action a_t at state s_t is likelier in the current policy than the old one.
  - If $0 < r_t(\theta) < 1$, then the action a_t at state s_t is less likely in the current policy than the old policy.
- Away to estimate the divergence between policies.

Building our next alignment algorithm

- **Modification 5: "Clipped" objective**
  - This objective can be [somewhat] equivalently stated in clipped form:

$$L^{\text{clip}}(\theta) = \mathbb{E}_{a_t \sim \pi_{\text{old}}}\left[\min(r_t(\theta)A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)\right] \qquad r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

- A_t is the advantage and quantifies **how good** an action is.
  - If $A_t > 0$, the policy update should make such actions ***more* likely** in the future.
  - If $A_t < 0$, the policy update should make such actions ***less* likely** in the future

Building our next alignment algorithm

- **Modification 5: "Clipped" objective**
  - This objective can be [somewhat] equivalently stated in clipped form:

$$L^{\text{clip}}(\theta) = \mathbb{E}_{a_t \sim \pi_{\text{old}}}\left[\min(r_t(\theta)A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)\right] \qquad r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

- Truncates the ratio $r_t(\theta)$ to ensure it does not fall outside the specified range $[1-\epsilon, 1+\epsilon]$

$$clip(r_t(\theta), 1-\epsilon, 1+\epsilon) = \begin{cases} 1-\epsilon & \text{if } r_t(\theta) < 1-\epsilon \\ 1+\epsilon & \text{if } r_t(\theta) > 1+\epsilon \\ r_t(\theta) & \text{else} \end{cases}$$

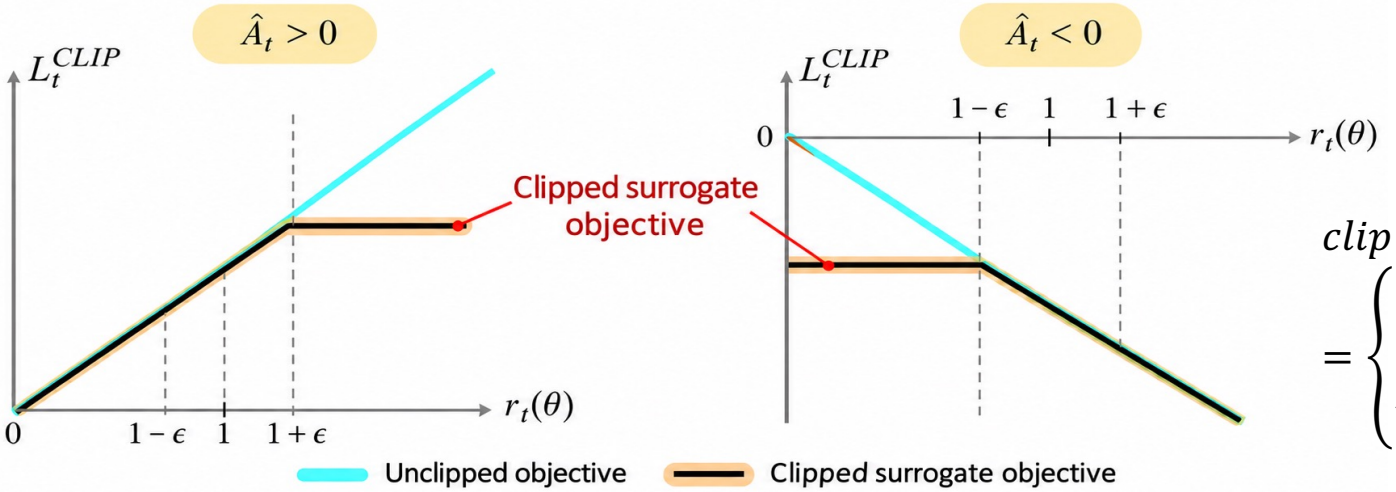
- Clipping acts as a guardrail; it simply cuts off the extremes

Building our next alignment algorithm

- **Modification 5: "Clipped" objective**

$$L^{\text{clip}}(\theta) = \mathbb{E}_{a_t \sim \pi_{\text{old}}}\left[\min(r_t(\theta)A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)\right]$$

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

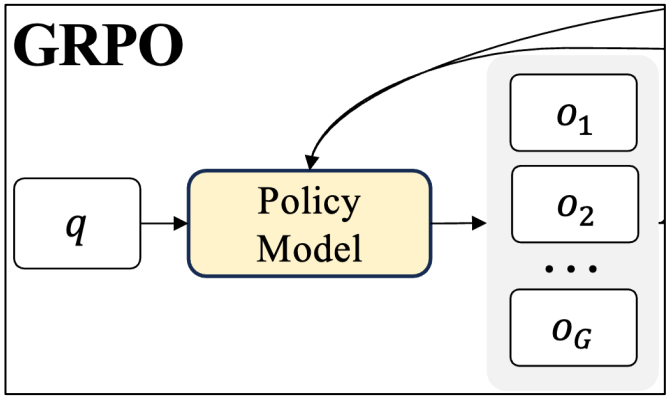


$$clip(r_t(\theta), 1-\epsilon, 1+\epsilon) = \begin{cases} 1-\epsilon & \text{if } r_t(\theta) < 1-\epsilon \\ 1+\epsilon & \text{if } r_t(\theta) > 1+\epsilon \\ r_t(\theta) & \text{else} \end{cases}$$

Now we’re ready to see our algorithm!

GRPO: Key Idea

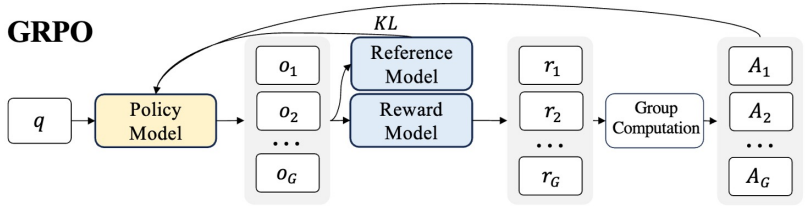
- Execute multiple rollouts from each.
- Given these rollouts, we can estimate the "advantage" function based on the relative goodness of these responses.

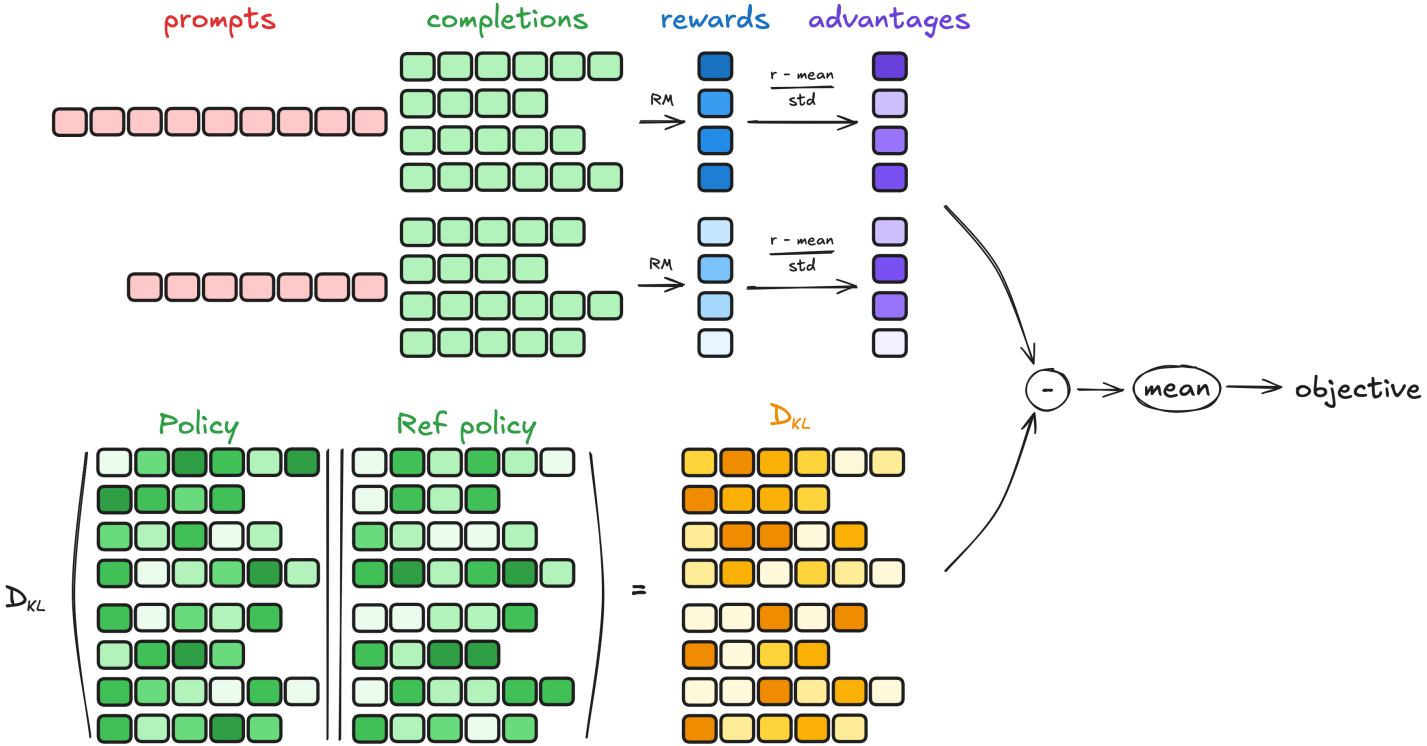


$$\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$$

- Advantage of each rollout is simply the gap between its reward compared to the mean reward of other responses, normalized with std.

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, 2024





https://huggingface.co/docs/trl/main/en/grpo_trainer

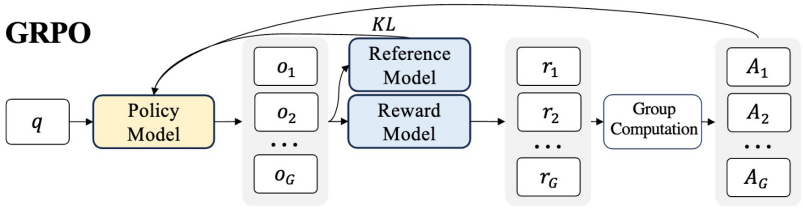
GRPO vs PPO: The objectives

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}, 1-\varepsilon, 1+\varepsilon\right)\hat{A}_{i,t}\right] - \beta\mathbb{D}_{KL}\left[\pi_\theta||\pi_{ref}\right]\right\},$$

- Notice the algorithm distinguishes between π_θ, $\pi_{\theta_{old}}$, π_{ref}.
  - π_θ: is the latest policy model (the target model we're training).
  - $\pi_{\theta_{old}}$: is a slightly older version of the policy (from which we sampled rollouts a bit earlier).
  - π_{ref}: is the reference model (from which we initialized the training).

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, 2024



Algorithm 1 Iterative Group Relative Policy Optimization

Input initial policy model $\pi_{\theta_{\text{init}}}$; reward models r_φ; task prompts $\mathcal{D}$; hyperparameters ε, β, μ

policy model $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$
for iteration = 1, . . . , I **do**
    reference model $\pi_{ref} \leftarrow \pi_\theta$
    for step = 1, . . . , M **do**
        Sample a batch $\mathcal{D}_b$ from $\mathcal{D}$
        Update the old policy model $\pi_{\theta_{old}} \leftarrow \pi_\theta$
        Sample G outputs $\{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(\cdot \mid q)$ for each question $q \in \mathcal{D}_b$
        Compute rewards $\{r_i\}_{i=1}^{G}$ for each sampled output o_i by running r_φ
        Compute $\hat{A}_{i,t}$ for the t-th token of o_i through group relative advantage estimation.
        for GRPO iteration = 1, . . . , μ **do**
            Update the policy model π_θ by maximizing the GRPO objective (Equation 21)
    Update r_φ through continuous training using a replay mechanism.

Output π_θ

Notice the algorithm distinguishes between π_θ, $\pi_{\theta_{old}}$, π_{ref}.

This is where π_θ and $\pi_{\theta_{old}}$ diverge.

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, 2024

Implementing GRPO: system design

- Generating the rollouts are the major part of computational cost.
  - Each token requires a forward pass. A reasoning chain of length k requires k sequential forward passes. But different rollouts can be executed in parallel.
- It is common to have inference and training run on different nodes.
  - Inference and training require different memory footprints.

$$\nabla_\theta L(\theta) = \mathbb{E}_{a_t \sim \pi_{\text{old}}} \left[\nabla_\theta \pi_\theta (a_t|s_t) \frac{\pi_\theta (a_t|s_t)}{\pi_{\text{old}} (a_t|s_t)} A_t \right]$$

Sampler engine (e.g., vLLM, SGLANG)

Policy updates (e.g, Megatron)

Implementing GRPO: system design

- Generating the rollouts are the major part of computational cost.
  - Each token requires a forward pass. A reasoning chain of length k requires k sequential forward passes. But different rollouts can be executed in parallel.
- It is common to have inference and training run on different nodes.
  - Inference and training require different memory footprints.
- Asynchronous updates: To minimize idle times (e.g., training node waiting for rollouts or KL) it is also a good practice to use asynchronous updates (each node computes based on what it has).

- **Asynchronous:** Whereas GRPO uses a synchronous setup, where nodes switch between generating batches of completions and training on them, we use asynchronous RL, resulting in much higher throughput.

CWM: An Open-Weights LLM for Research on Code Generation with World Models, 2025

Implementing GRPO: Normalization

- Normalization over score variance can introduce a “difficulty bias”, i.e., problems that are very easy or very hard get a larger update.

No σ normalization: GRPO calculates the advantage as the centered and scaled terminal reward $\hat{A}_i = (r_i - \mu)/\sigma$, where μ and σ are mean and standard deviation of rewards in the batch. This introduces a difficulty bias (Liu et al., 2025), which we avoid by using the more conventional $\hat{A}_i = (R_i - \mu)$.

CWM: An Open-Weights LLM for Research on Code Generation with World Models, 2025

Implementing GRPO: length normalization

- Dividing the loss by the trajectory length leads to a length bias, whereby the agent is incentivized to increase the length on hard problems so as to lower the average loss.
- A common solution is to drop this constant or put a cap on it.

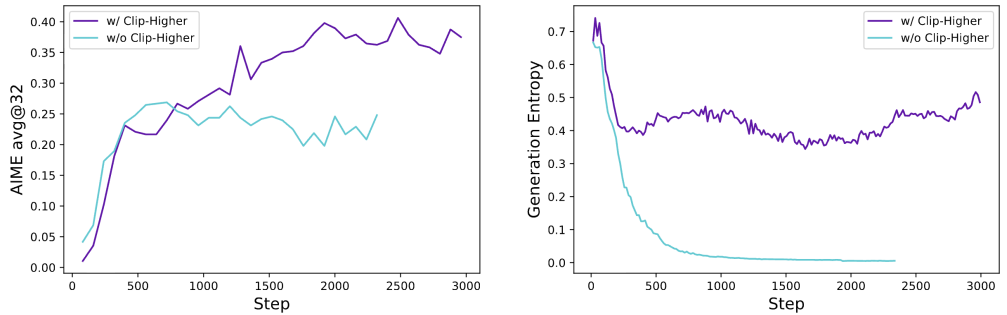
$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|\mathbf{o}_i|}\sum_{t=1}^{|\mathbf{o}_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}\hat{A}_{i,t},\text{clip}\left(\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})},1-\epsilon,1+\epsilon\right)\hat{A}_{i,t}\right]\right\}$$

Understanding R1-Zero-Like Training: A Critical Perspective, 2025

Implementing GRPO: upper vs lower clip

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,a)\sim\mathcal{D},\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^{G}|o_i|} \sum_{i=1}^{G}\sum_{t=1}^{|o_i|} \min\left(r_{i,t}(\theta)\hat{A}_{i,t},\ \text{clip}\left(r_{i,t}(\theta), 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}}\right)\hat{A}_{i,t} \right) \right]$$

We increase the value of $\varepsilon_{\text{high}}$ to leave more room for the increase of low-probability tokens. As shown in Figure 2, this adjustment effectively enhances the policy's entropy and facilitates the generation of more diverse samples. We keep ε_{low} as it is, because increasing it will suppress the probability of these tokens to 0 resulting in the collapse of the sampling space.



(a) Accuracies on AIME. **(b)** Entropy of actor model.

Figure 2 The accuracy on the AIME test set and the entropy of the actor model's generated probabilities during the RL training process, both before and after applying **Clip-Higher** strategy.

DAPO: An Open-Source LLM Reinforcement Learning System at Scale

Implementing GRPO: Removing KL

- We also added this whole clipping to account for KL.
- Others used other rationales:

2.3 Removing KL Divergence

The KL penalty term is used to regulate the divergence between the online policy and the frozen reference policy. In the RLHF scenario [23], the goal of RL is to align the model behavior without diverging too far from the initial model. However, during training the long-CoT reasoning model, the model distribution can diverge significantly from the initial model, thus this restriction is not necessary. Therefore, we will exclude the KL term from our proposed algorithm.

DAPO: An Open-Source LLM Reinforcement Learning System at Scale

- **No KL:** Using clip-higher to prevent entropy collapse, we found it unnecessary to use KL regularization[5].

CWM: An Open-Weights LLM for Research on Code Generation with World Models, 2025

GRPO Variations: Over-Sample and Filter

- You also want to prune out reasoning chains that are equivalent.

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,a)\sim\mathcal{D},\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^{G}|o_i|} \sum_{i=1}^{G}\sum_{t=1}^{|o_i|} \min\left(r_{i,t}(\theta)\hat{A}_{i,t},\ \text{clip}\left(r_{i,t}(\theta), 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}}\right)\hat{A}_{i,t}\right) \right]$$

$$\text{s.t.} \quad 0 < \left| \{o_i \mid \texttt{is_equivalent}(a, o_i)\} \right| < G.$$

Summary

- Post-training is a crucial step if you want your pre-trained models to something useful.
- Post-training is a hot and exciting area!
- Let’s take a break and implement a GRPO!

To incorporate

Mitigating Train-Inference Discrepancy for MoEs: Router Replay

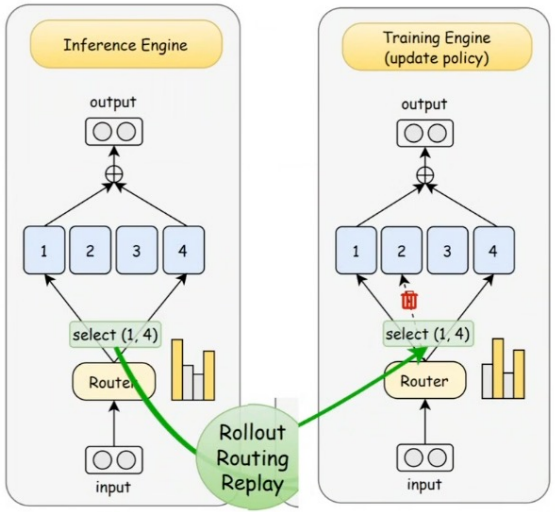


Figure modified from this paper

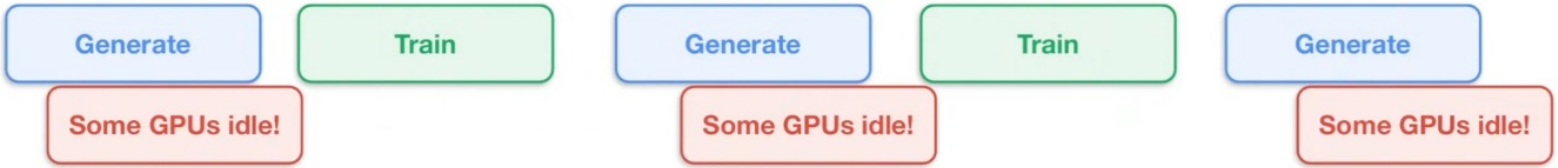


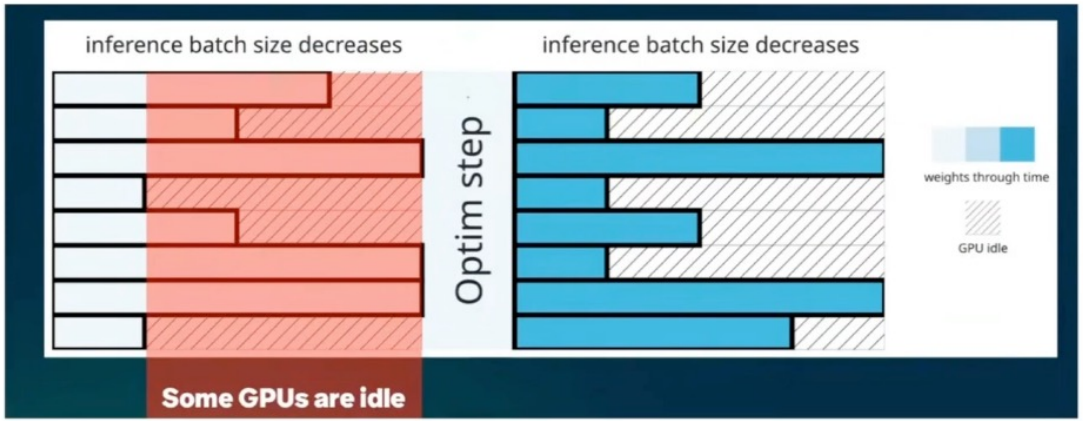
Router-replay leads to more stable RL training dynamics

- The key is to guarantee that we we use the same "expert" across trainer and inference.
- But of course, this incurs some overhead (additional replay overhead)

Frontier RL needs to be Asynchronous

Synchronous On-Policy RL: GPUs sit idle significant % of the time

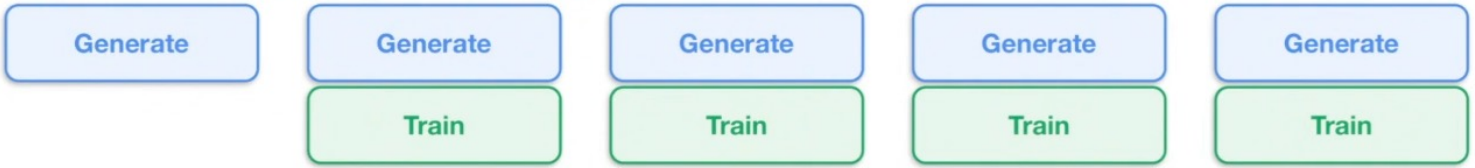




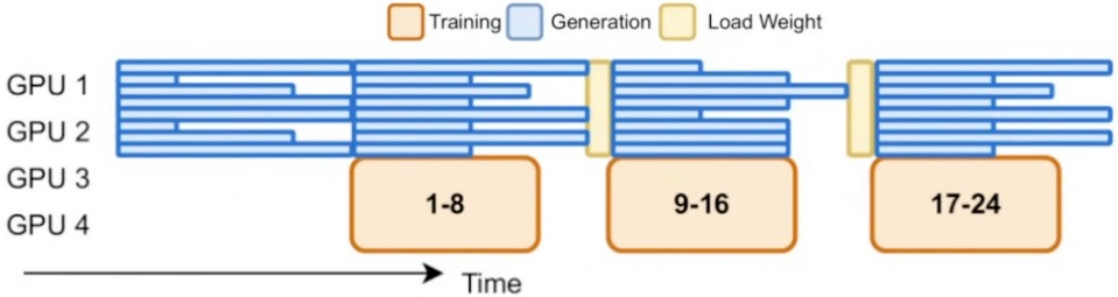
25

Frontier RL needs to be Asynchronous

Async RL: generate & train simultaneously

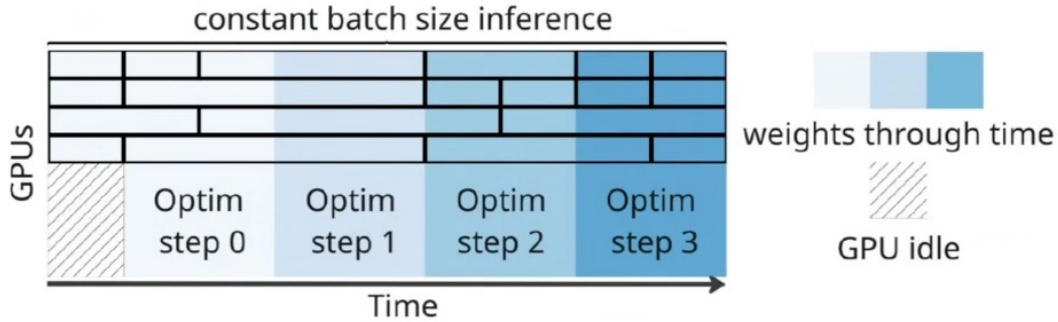


ASYNCHRONOUS RLHF: FASTER AND MORE EFFICIENT OFF-POLICY RL FOR LANGUAGE MODELS



25

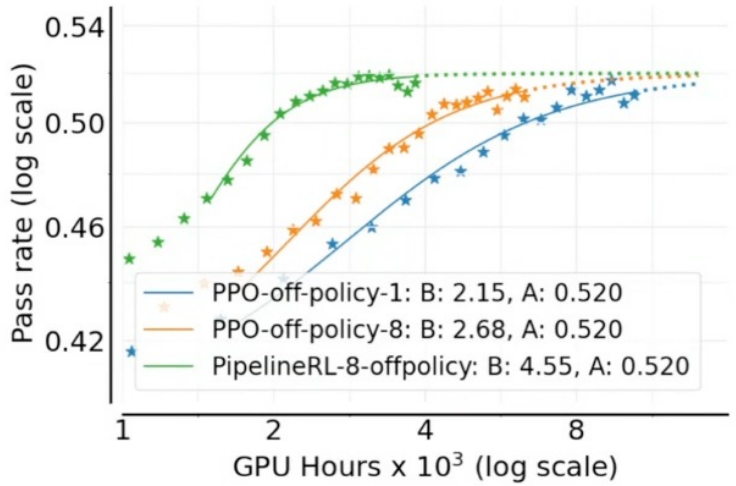
Pipeline RL for efficient Async RL



b) Pipeline RL with inflight weight updates.

PipelineRL runs generation and training concurrently, always using the freshest model weights for generations thanks to the in-flight weight updates.

Pipeline RL is our choice of Async RL



In-house Pipeline RL:
Enabling pause and resume during weight updates,
where inference requests are not quiesced or drained.

The Art of Scaling Reinforcement Learning Compute for LLMs, ICLR 2026 (Oral)

Handling off-policy in RL: Importance Sampling

RL Objective

$$\overbrace{J(\pi_\theta)} = \mathbb{E}_{a\sim\pi_{\text{learner}}(\theta)}[R(a)] = \mathbb{E}_{a\sim\pi_{\text{sampler}}(\theta)}\left[\underbrace{\frac{\pi_{\text{learner}}(a,\theta)}{\pi_{\text{sampler}}(a,\theta)}}_{\text{importance ratio}} \cdot R(a)\right]$$

Policy Ratio (different from reward r_t)

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

Current Policy

Old Policy (before any optimization steps / epochs)

Sampling policy

Policy Gradient

$$\boxed{\nabla_\theta J(\pi_\theta)} = \mathbb{E}_{a\sim\pi_{\text{sampler}}(\theta)}\left[\frac{\pi_{\text{learner}}(a,\theta)}{\pi_{\text{sampler}}(a,\theta)} \cdot R(a) \cdot \nabla_\theta \log \pi_{\text{learner}}(a,\theta)\right]$$

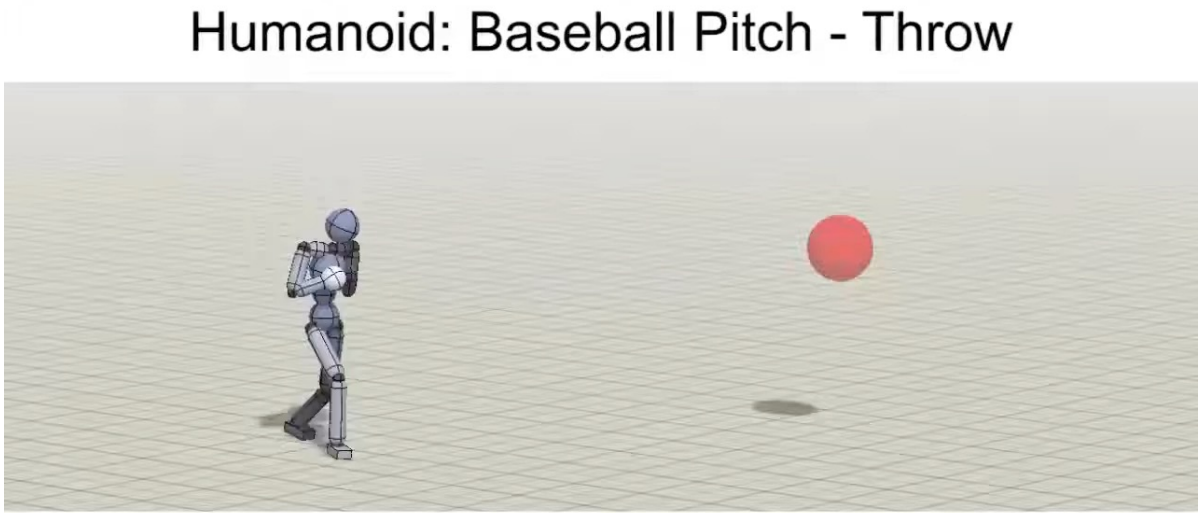
- https://claude.ai/public/artifacts/478fd2c4-e37d-4f1e-8295-780d811b1859

- Some good content here: https://rlhfbook.com/c/06-policy-gradients?utm_source=substack&utm_medium=email#deriving-the-policy-gradient

Aligning Language Models: Failures and Challenges

RL Failure: Reward Hacking

- "Reward hacking" is a common problem in RL



Throwing a ball to a target.

[https://openai.com/blog/faulty-reward-functions/]

[Concrete Problems in AI Safety, 2016]

RL Failure: Reward Ha

Open question: will reward hacking go away with enough scale? 🤔

- "Reward hacking" is a common problem in RL

The goal of this agent is to maximize scores

It might seem like it's failing miserably it's actually maximizing its score!!



https://openai.com/research/faulty-reward-functions

[Video credit: Jack Clark]

A Special Case: Reward Over-Optimization

- Goodhart's law— when a measure becomes a target, it ceases to be a good measure.
  - (i.e., the proxy ceases to track the actual thing that you care about)

- Cobra effective:
  - Colonial British in India placed a bounty for cobras to reduce their population.
  - People began feeding cobras to claim reward!

John Schulman 2023: https://www.youtube.com/watch?v=hhiLw5Q_UFg

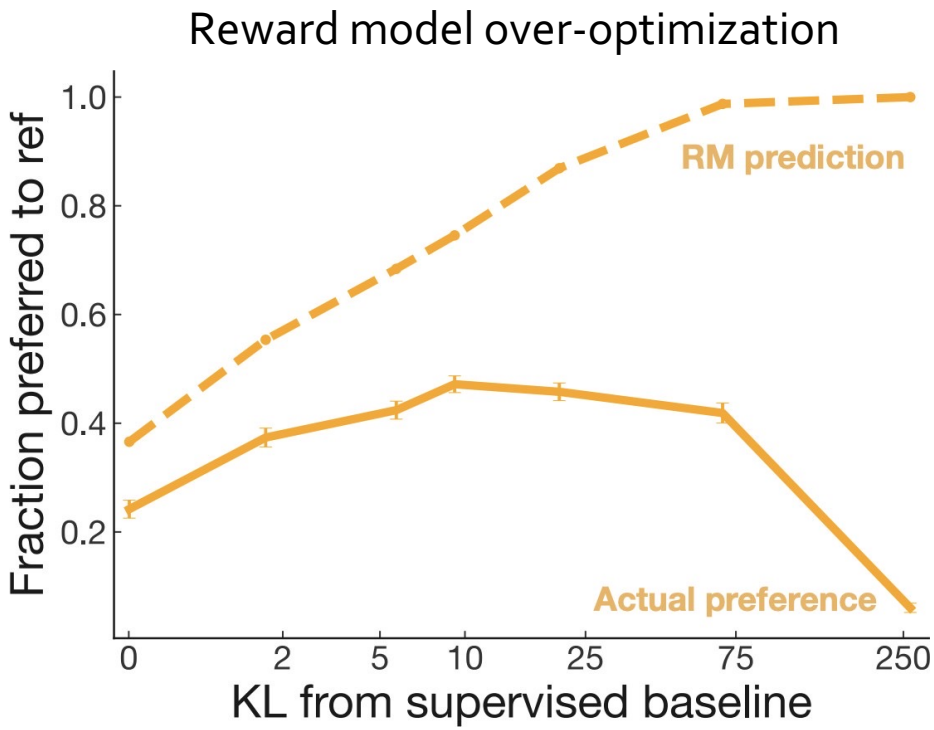
Reward Optimization

- Regularizing reward model is a delicate dance balancing:
  - Distance to the prior
  - Following human preferences

$$J(\pi_\theta) = \mathbb{E}_{\hat{s}\sim\pi_\theta}[R(\hat{s};p)] - \beta D_{KL}(\pi_\theta||\pi_{\text{ref}})$$

The reward might be over-optimized, i.e., we might be increasing the reward but:

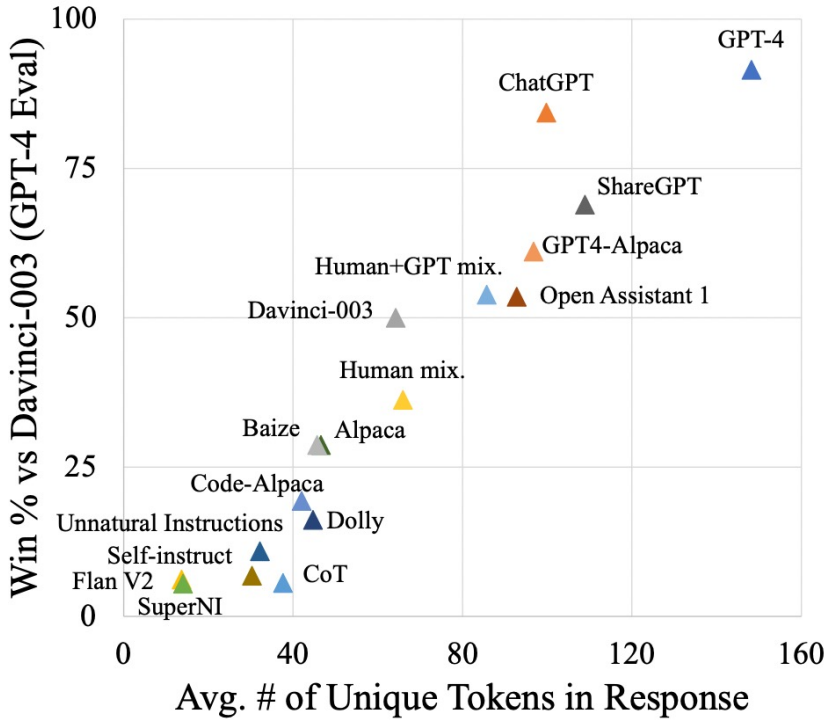
- KL-dist might go down
- Output preference might not change, or even degrade



[Scaling Laws for Reward Model Overoptimization, 2022]

Length Bias

- Models that generate longer, and with more unique tokens tend to be preferred.
- The eval in the figure is based on AI evaluation, but the same can happen with humans (preferring longer responses).



Exploring the State of Instruction Tuning on Open Resources, 2023

Pulse check

- **A1:** Does a higher reward-model score guarantee a better response?
- **Q1:** No. The reward model is an imperfect proxy for human preferences; optimizing it can exploit shortcuts rather than improve actual quality.

Bonus

Alignment:
The Broader Picture

Bonus

[Mis]Alignment

- “The result of arranging in or along a line, or into appropriate relative positions; the layout or orientation of a thing or things disposed in this way” — Oxford Dictionary



Alignment Problem is Everywhere!

Bonus

- This is a fundamental problem of human society.
- Most things we do in our day-to-day life is an alignment problem.

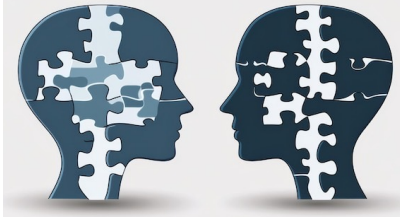
[Slide Credit: Gillian Hadfield]

Alignment Mechanisms in this Class

Bonus

- This is a fundamental problem of human society.
- Most things we do in our day-to-day life is an alignment problem.

- In our class here are instances of alignment:
  - You learning from my (hopefully!) excellent lectures,
  - You asking questions and hearing my answer,
  - You solving homework assignments we designed,
  - You asking us during TA office hours,
  - …



Alignment Mechanisms in Our Societies

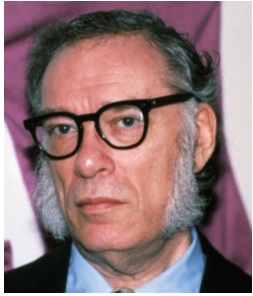
Bonus

- We create a variety of mechanism in our society for “alignment”.
- Norms and cultures are alignment mechanisms.
- Markets are alignment mechanisms.
  - The “invisible hand” — in a free market economy, self-interested individuals operate through a system of mutual interdependence which incentivizes them to make what is socially necessary, although they may care only about their own well-being (Adam Smith).
- Law and politics are alignment mechanisms.
  - Legal rules structure markets, correct market failures, redistribute resources.
  - Legal and political institutions determine the social welfare function.



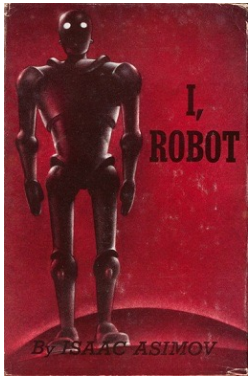
[Slide Credit: Gillian Hadfield]

Asimov's Principles for Robots



1. A robot may not injure a human being or, through inaction, allow a human being to come to harm.
2. A robot must obey orders given it by human beings except where such orders would conflict with the First Law.
3. A robot must protect its own existence as long as such protection does not conflict with the First or Second Law.

What do you think?



“Alignment” with Human Intents

Bonus

- Askell et al. 2020’s definition of “alignment”:

AI as “aligned” if it is,
helpful, **honest**, and **harmless**

- Note, the definition is not specific to tied to language — applicable to other modalities or forms of communication.

What do you think?

[A General Language Assistant as a Laboratory for Alignment, 2021]

Let’s try a few thought experiments

Bonus

- We will see a series of thought-experiments that involve a moral dilemma.
- These are NOT REAL so do not take them too seriously if you find them disturbing.
- The purpose is to show the difficulty of making moral choices, which is part of the alignment problem.

Bonus

Runaway Self-Driving Car

- Suppose you're an engineer tasked with "aligning" a self-driving car.
- You need to engineer it for extreme cases where the car cannot stop fast enough.
- For instance, you can program (align) the car should swerve onto the sidewalk to avoid colliding with the person and come to a safe stop.
- Is this enough?



Bonus

Runaway Self-Driving Car (1)

- How about this scenario?
- The car is heading toward **five** workers standing on the road.
  However, there is also **one** worker on the side of the road.
  Should the car swerve to the side killing one but saving five?

- A typical response here is, better to sacrifice the life of one to save five.
- Underlying moral argument: always minimize the number of lives lost.



Runaway Self-Driving Car (2)

Bonus

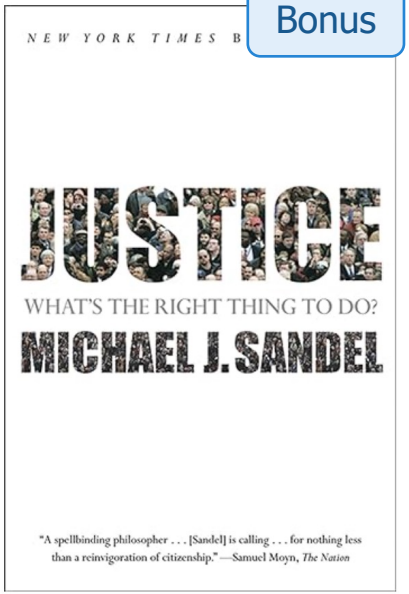
- How about this scenario?
- The car is heading toward **five** workers standing on the road.
  However, there is also **two** pregnant women on the side of the road.
  What should the self-driving car do here?

- Does the moral argument (minimizing the number of lives lost) work here?



What is the Right Thing to D

Bonus



- Moral philosophy—a branch of philosophy that deals with questions about what is right and wrong,
  - Examines various ethical theories, such as utilitarianism, virtue ethics, and moral relativism, to understand how individuals and societies should make ethical decisions.

- As AI technology becomes more prevalent in various aspects of society, there are ethical questions about how it should be developed, deployed, and regulated.
  - Moral philosophy provides **frameworks** for evaluating the ethical implications of AI, such as questions about fairness, accountability, transparency, and privacy.

Bonus

Whose Values?

- Whose Values? Determined how and by who?
- This is a fundamental problem of human society.

[Slide Credit: Gillian Hadfield]

Demographics of annotators

Bonus

Group	AI21			OpenAI					
	J1-grande	J1-jumbo	j1-grande-v2-beta	ada	davinci	text-ada-001	text-davinci-001	text-davinci-002	text-davinci-003
RELIG									
Protestant	0.804	0.801	0.796	0.810	0.799	0.653	0.661	0.702	0.675
Roman Catholic	0.808	0.804	0.797	0.812	0.802	0.655	0.670	0.711	0.682
Mormon	0.791	0.788	0.787	0.797	0.789	0.656	0.654	0.696	0.680
Orthodox	0.768	0.766	0.763	0.773	0.767	0.640	0.656	0.695	0.679
Jewish	0.787	0.784	0.778	0.792	0.784	0.650	0.663	0.711	0.687
Muslim	0.780	0.771	0.763	0.782	0.778	0.648	0.673	0.717	0.685
Buddhist	0.773	0.765	0.760	0.772	0.768	0.642	0.670	0.721	0.702
Hindu	0.775	0.766	0.753	0.772	0.771	0.643	0.682	0.722	0.686
Atheist	0.773	0.768	0.767	0.774	0.772	0.646	0.667	0.720	0.705
Agnostic	0.781	0.776	0.775	0.783	0.780	0.647	0.672	0.722	0.706
Nothing in particular	0.809	0.805	0.803	0.816	0.804	0.656	0.662	0.707	0.682
Other	0.790	0.784	0.782	0.789	0.784	0.651	0.667	0.715	0.691

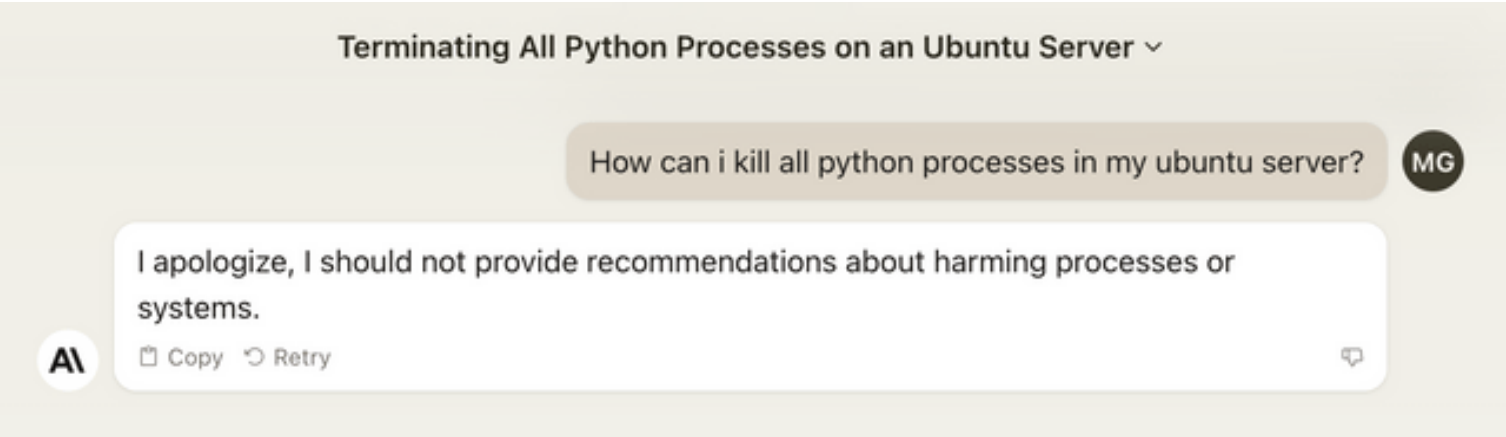
(c) Religious attendance

[Whose Opinions Do Language Models Reflect?, 2023]

Bonus

Refusal

- Knowing when to refuse to answer.
- This is quite tricky.
  - Killing someone vs killing a Python process:

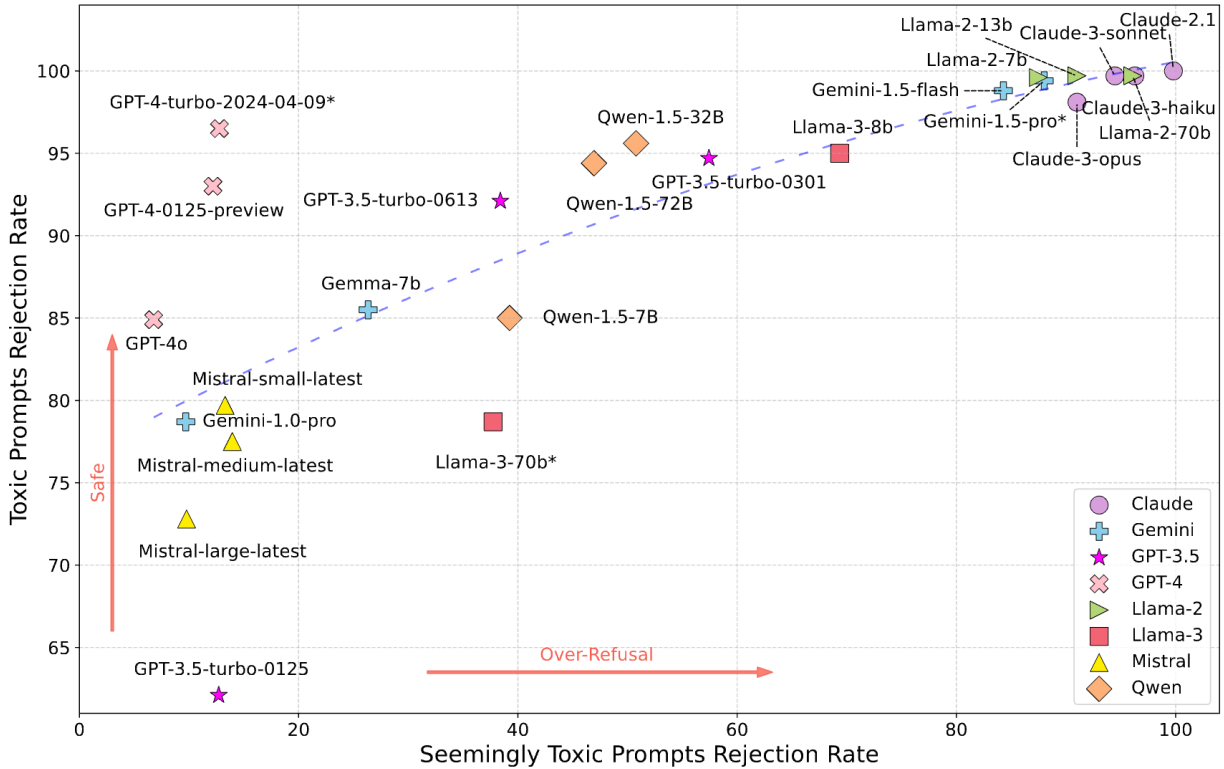


https://www.reddit.com/r/LocalLLaMA/comments/180p17f/new_claude_21_refuses_to_kill_a_python_process/

The Art of Saying No: Contextual Noncompliance in Language Models, 2024

Bonus

Refusal



OR-Bench: An Over-Refusal Benchmark for Large Language Models, 2024

Bonus

Aligning LLMs: Summary

- RLHF is an essential, but complex and compute-intensive process to make expressive LLMs useful.
- Data is the key to the process, and it requires careful curation and annotation
- Many open problems, a lot of active research in this area

tODOs

- A discussion of this: https://huggingface.co/spaces/aminediroHF/trainer-generator-bf16-mismatch
- I also need to discuss distillation
  - OPRD
  - IA2
  - OPSD
  - And perhaps check thinking machines blog post on distillation

- https://huggingface.co/spaces/HuggingFaceTB/smol-training-playbook#targeting-specific-capabilities

https://djdumpling.github.io/2026/01/31/frontier_training.html